Entwicklung einer Softwarelösung zur Bestimmung von Gipfeln auf topografischen Karten basierend auf vorangegangenen Projektergebnissen

Studienarbeit

an der Dualen Hochschule Baden-Württemberg Stuttgart

von

Jinho An, Philipp Böhm

Abgabedatum	12. Juni 2025
Studiengang	Informatik
Bearbeitungszeitraum	13. Januar 2025 - 12. Juni 2025
Matrikelnummer, Kurs	3084111, 3258299, 22IN
Ausbildungsfirma	Siemens AG, Stuttgart
Betreuer	Prof. Dr. Roman Gruden

Erklärung

Ich erkläre hiermit ehrenwörtlich:

1. dass ich meine Studienarbeit mit dem Thema *Entwicklung einer Softwarelösung zur Bestimmung von Gipfeln auf topografischen Karten basierend auf vorangegangenen Projektergebnissen* ohne fremde Hilfe angefertigt habe;
2. dass ich die Übernahme wörtlicher Zitate aus der Literatur sowie die Verwendung der Gedanken anderer Autoren an den entsprechenden Stellen innerhalb der Arbeit gekennzeichnet habe;
3. dass ich meine Studienarbeit bei keiner anderen Prüfung vorgelegt habe;
4. dass die eingereichte elektronische Fassung exakt mit der eingereichten schriftlichen Fassung übereinstimmt.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Stuttgart, 12. Juni 2025

Jinho An, Philipp Böhm

Kurzreferat

Diese Studienarbeit beschreibt die Entwicklung einer Software zur automatischen Identifikation von Gipfeln auf Basis digitaler Höhenmodelle (GeoTIFF). Die Software kombiniert klassische Algorithmen wie den Bresenham- und Dijkstra-Algorithmus mit modernen Filtertechniken zur effizienten Bestimmung geowissenschaftlicher Merkmale wie Prominenz und Dominanz. Grundlage sind etablierte Klassifikationssysteme für Gipfel, die objektive Kriterien verwenden. Ziel ist die Entwicklung eines robusten, wiederholbaren und nutzerfreundlichen Systems zur Gipfelerkennung, das eine visuelle Auswertung sowie weiterführende Forschung in der Topografieanalyse unterstützt.

Schlüsselwörter

Bresenham-Algorithmus, Dijkstra-Algorithmus, Digitales Höhenmodell (DHM), Dominanz, GeoTIFF, Gipfel, Orografie, Prominenz, Python, Topografie.

Abstract

This study presents the development of a software solution for the automatic identification of mountain summits based on digital elevation models (GeoTIFF). The approach combines established algorithms such as Bresenham's and Dijkstra's with modern filtering techniques to efficiently compute geoscientific features like prominence and dominance. The work is grounded in standardized classification systems using objective summit criteria. The goal is to create a robust, reproducible, and user-friendly system for peak detection, supporting both visual analysis and further research in topographic data processing.

Keywords

Bresenham's algorithm, Dijkstra's algorithm, Digital Elevation Model (DEM), Dominance, GeoTIFF, Summit, Orography, Prominence, Python, Topography.

Inhaltsverzeichnis

Abkürzungsverzeichnis **VII**

Abbildungsverzeichnis **IX**

Quellcodeverzeichnis **X**

Tabellenverzeichnis **XI**

1 Einleitung **1**
1.1 Einführung . . . 1
1.2 Problemstellung und Zielsetzung . . . 2
1.3 Methodische Vorgehensweise . . . 3

2 Theoretische Grundlagen **4**
2.1 Geowissenschaften . . . 4
2.1.1 Überblick . . . 4
2.1.2 Disziplinen . . . 4
2.1.3 Grundbegriffe . . . 6
2.2 Das digitale Höhenmodell . . . 9
2.2.1 Definition . . . 9
2.2.2 Varianten . . . 9
2.2.3 Erfassung der Höhendaten . . . 10
2.2.4 Modellierung der Höhendaten . . . 12
2.2.5 Einflussfaktoren auf die Genauigkeit . . . 14
2.2.6 GeoTIFF . . . 15
2.3 Kriterien zur Klassifizierung von Gipfeln . . . 18
2.3.1 Höhe . . . 18
2.3.2 Prominenz . . . 19
2.3.3 Dominanz . . . 21
2.3.4 Orografische Dominanz . . . 22
2.4 Schwellwerte zur Klassifizierung von Gipfeln . . . 23
2.4.1 Union Internationale des Associations d'Alpinisme (UIAA) . . . 23
2.4.2 Das erweiterte Prominenzkonzept . . . 23
2.4.3 Eigenständigkeitsklassen . . . 24

3 Stand der Technik **26**
3.1 Ansätze zur Gipfelfindung . . . 26
3.1.1 Filterung nach Prominenz und Dominanz . . . 26
3.1.2 Heuristische Filterung nach lokaler Umgebung . . . 27
3.1.3 Multiskalige Fuzzy-Klassifizierung . . . 27
3.2 Bestimmung potenzieller Gipfel . . . 29
3.2.1 Morphologische Operationen . . . 29
3.2.2 Lokaler Maximum-Filter . . . 32
3.2.3 Vergleich von Morphologische Operation und Maximum-Filter 33
3.3 Algorithmen zur Berechnung der Prominenz . . . 34
3.3.1 Bresenham-Algorithmus . . . 34
3.3.2 Dijkstra-Algorithmus . . . 36
3.3.3 Vergleich zwischen Bresenham- und Dijkstra-Algorithmus . 38
3.4 Algorithmen zur Berechnung der Dominanz . . . 41
3.5 Fazit . . . 42

4 Implementierung der automatischen Gipfelerkennung **43**
4.1 Anforderungen . . . 43
4.1.1 Funktionale Anforderungen . . . 43
4.1.2 Nichtfunktionale Anforderungen . . . 44
4.2 Auswahl der Technologien . . . 45
4.2.1 Programmiersprache . . . 45
4.2.2 Bibliotheken für die Geodatenverarbeitung . . . 45
4.2.3 Bibliotheken für die Gipfelberechnung . . . 46
4.2.4 Bibliotheken für die grafische Benutzeroberfläche . . . 47
4.3 Architektur . . . 49
4.4 Implementierung der Datenextraktion (reader.py) . . . 51
4.4.1 Auslesen der Rasterdaten . . . 51
4.4.2 Auslesen der Metadaten . . . 52
4.5 Implementierung der Benutzeroberfläche (gui.py) . . . 53
4.5.1 Überblick . . . 53
4.5.2 Fensterstruktur des Hauptdialogs . . . 54
4.5.3 Platzierung der GUI-Elemente . . . 55
4.5.4 Hochladen der DHMs . . . 57
4.5.5 Darstellung der DHMs . . . 57
4.5.6 Darstellung der berechneten Gipfel . . . 59
4.5.7 Ergebnis . . . 61
4.6 Implementierung der Geofunktionen (geo_utils.py) . . . 62
4.6.1 Umrechnung der Weltkoordinaten in WGS84-Koordinaten . . 62
4.6.2 Umrechnung der Auflösung in Pixel pro Meter . . . 63

4.7 Implementierung der Gipfelfindung (peak_analysis.py) 65
4.7.1 Überblick 65
4.7.2 Bestimmung der potenziellen Gipfel 66
4.7.3 Berechnung der Prominenz 68
4.7.4 Berechnung der Dominanz 77
4.7.5 Berechnung der orografischen Dominanz 80
4.7.6 Ergebnis und Evaluierung 80
4.8 Fazit 82

5 Zusammenfassung und Ausblick **83**
5.1 Zusammenfassung 83
5.2 Ausblick 84

Literaturverzeichnis **VIII**

Abkürzungsverzeichnis

CRS	Coordinate Reference System
DEM	Digital Elevation Model
DGM	Digitales Geländemodell
DHM	Digitales Höhenmodell
DOM	Digitales Oberflächenmodell
DSM	Digital Surface Model
DTM	Digital Terrain Model
EPSG	European Petroleum Survey Group Geodesy
GDAL	Geospatial Data Abstraction Library
GIS	Geoinformationssystem
GPS RTK	Global Positioning System Real-Time Kinematic
GUI	Grafical User Interface
IDW	Inverse Distance Weighting
JIT	Just-In-Time
IfSAR	Interferometric Synthetic Aperture Radar
LIDAR	Light Detection and Ranging
NumPy	Numerical Python
RADAR	Radio Detection and Ranging
RGB	Rot Grün Blau
SciPy	Scientific Python
TIFF	Tagged Image File Format

TIN	Triangulated Irregular Network
TS	Totalstation
UIAA	Union Internationale des Associations d'Alpinisme

Abbildungsverzeichnis

2.1 Geometrie eines Sattelpunkts 7
2.2 Unterschied zwischen DGM und DOM 9
2.3 Beispiel für die Multiband-Unterstützung bei GeoTIFF [23] 16
2.4 Höhendefinitionen im Überblick [16] 18
2.5 Darstellung der Prominenz 20
2.6 Darstellung der Dominanz 21

3.1 Beispielsergebnis der Anwendung der Erosion 29
3.2 Beispielsergebnis der Anwendung der Dilatation 30
3.3 Beispielsergebnis der Anwendung der kombinierten morphologischen Operationen 31
3.4 Beispielsergebnis der Anwendung des Maximum-Filters 32
3.5 Beispiel für eine Linie nach dem Bresenham-Algorithmus 35
3.6 Beispiel eines Pfades nach dem Dijkstra-Algorithmus 37
3.7 Beispiel eines Pfades nach dem Max-Min-Dijkstra-Algorithmus . . 38
3.8 Abweichung der Prominenzberechnung mit dem Bresenham-Algorithmus 39
3.9 Binäre Maske und die korrespondierende euklidische Distanztransformation [32] 41

4.1 Aufbau des Programms (nur wesentliche Funktionen) 49
4.2 Designentwurf für die GUI 53
4.3 2D- und 3D-Darstellung der berechneten Gipfel 60
4.4 GUI-Fenster der Anwendung zur Gipfelerkennung 61
4.5 Vergleich der Dauer der Prominenzberechnung mit und ohne Optimierung 77
4.6 Vergleich der berechneten Gipfel in der Wallis-Region 81

Quellcodeverzeichnis

4.1 Auslesen der Höhendaten aus der GeoTIFF 51
4.2 Auslesen der Metadaten aus der GeoTIFF 52
4.3 Implementierung der Fensterstruktur in CustomTkinter 54
4.4 Implementierung des Upload-Buttons in CustomTkinter 55
4.5 Erstellung der Tabelle zur Anzeige der Gipfelinformationen 56
4.6 Funktion zum Hochladen der GeoTIFF-Datei 57
4.7 Plot-Funktion für die Kartenanzeige 58
4.8 Darstellung der berechneten Gipfel in der Tabelle 59
4.9 Funktion zur Konvertierung von Koordinaten nach WGS84 62
4.10 Funktion zur Umrechnung der Kartenauflösung in Pixel pro Meter . 63
4.11 Algorithmus zur Findung lokaler Maxima 67
4.12 Berechnung der Prominenz von Gipfeln 69
4.13 Berechnung des nächsthöheren Nachbarn 70
4.14 Berechnung des Pfades zwischen zwei Punkten mit Bresenham-Algorithmus 72
4.15 Berechnung der Prominenz mit dem Max-Min-Dijkstra-Algorithmus 74
4.16 Berechnung der Dominanzdistanz 78

Tabellenverzeichnis

2.1 Schwellwerte des erweiterten Prominenzkonzepts[4] [25] 24
2.2 Definition der Indizes der Eigenständigkeit [25] 25

1 Einleitung

1.1 Einführung

Die fortschreitende Digitalisierung geowissenschaftlicher Daten eröffnet neue Möglichkeiten für deren automatisierte Analyse und Auswertung. Vor allem die Verarbeitung von digitalen Höhenmodellen nimmt hier eine zentrale Rolle ein, da sie die Grundlage für zahlreiche topografische Analysen bildet. Digitale Höhenmodelle repräsentieren die Geländeform der Erdoberfläche als Rasterdaten, typischerweise im GeoTIF-Format. Besonders die Identifikation und Klassifikation von Gipfeln auf Basis solcher Modelle stellt ein komplexes Problem dar, das sowohl geowissenschaftliche als auch informatische Expertise erfordert. Die vorliegende Arbeit setzt an diesem interdisziplinären Schnittpunkt an und entwickelt eine Softwarelösung zur automatisierten Gipfelerkennung. Dabei werden etablierte Algorithmen aus der Bildverarbeitung und Graphentheorie auf digitale Geländemodelle angewandt, um charakteristische Merkmale wie Prominenz und Dominanz systematisch zu erfassen.

Die Arbeit knüpft an vorherige Projekte an, welche sich mit der geodätischen Analyse topografischer Daten befasst haben [1] [2] [3]. Sie verfolgt das Ziel, diese Ergebnisse zu konsolidieren und funktional zu erweitern. Im Zentrum stehen dabei sowohl die algorithmische Bestimmung potenzieller Gipfelpunkte als auch deren quantitative Bewertung auf Basis geowissenschaftlicher Klassifikationskriterien. Die entwickelten Methoden und ihre technische Umsetzung liefern einen praxisorientierten Beitrag zur digitalen Topografieverarbeitung und schaffen die Grundlage für weiterführende Anwendungen in der Kartografie, der geowissenschaftlichen Forschung sowie der automatisierten Landschaftsanalyse.

Die vorangegangenen Projektergebnisse sind ebenfalls im Literaturverzeichnis unter folgenden Verweisen zu finden:

1.2 Problemstellung und Zielsetzung

> „In der internationalen Diskussion gibt es keine klare Definition von einem „Berg“ oder „Gipfel“” [4]

Dieses Zitat von Eberhardt Jurgalski (2015), einer Koryphäe auf dem Gebiet der geografischen Gipfelklassifizierung, bringt das Hauptproblem der vorliegenden Arbeit auf den Punkt: Es gibt keine universell geltende Definition eines Gipfels. Das macht es so schwierig, aus vorhandenen, digitalen Höhenmodellen jene Punkte zu identifizieren, die nach objektiven Kriterien als Gipfel klassifiziert werden können. Die manuelle Erfassung und Bewertung von Gipfeln auf topografischen Karten ist sehr zeitaufwändig, fehleranfällig und erfordert fundiertes Expertenwissen. In der Praxis besteht ein hoher Bedarf an objektiven, wiederholbaren Verfahren zur automatisierten Gipfelerkennung, insbesondere für Anwendungen in der digitalen Kartografie und der geowissenschaftlichen Analyse.

Ziel der vorliegenden Arbeit ist daher die Entwicklung einer Softwarelösung, die Gipfel automatisiert identifizieren und klassifizieren kann. Die Eingabe der Höhendaten erfolgt dabei über digitale Höhenmodelle im GeoTIF-Format. Bei der Verarbeitung sollen etablierte Merkmale wie Höhe, Prominenz und Dominanz berücksichtigt werden. Die Lösung soll mithilfe algorithmischer Verfahren (Bresenham- und Dijkstra-Algorithmus) ein robustes Verfahren zur Berechnung typischer Klassifizierungsparameter wie der Prominenz und Dominanz integrieren und eine systematische Klassifikation gemäß gängiger Schwellenwerte durchführen. Neben der Entwicklung des algorithmischen Kerns steht die Integration in eine benutzerfreundliche grafische Oberfläche im Fokus. Die erarbeitete Lösung soll sowohl eine visuelle Darstellung der Ergebnisse ermöglichen als auch als Grundlage für weiterführende Forschungs- und Entwicklungsprojekte dienen.

1.3 Methodische Vorgehensweise

Die methodische Umsetzung gliedert sich in mehrere systematische Arbeitsschritte, die aufeinander aufbauen. Zu Beginn erfolgt eine Analyse existierender Ansätze zur Erkennung von Gipfeln, um geeignete Verfahren für die Umsetzung auszuwählen. Anschließend werden theoretische Grundlagen aus den Bereichen Geoinformatik, Bildverarbeitung und Graphentheorie herangezogen, um die verwendeten Methoden wissenschaftlich zu fundieren.

Die Softwareentwicklung beginnt mit der Extraktion von Raster- und Metadaten aus GeoTIFF-Dateien. Darauf folgt die Identifikation potenzieller Gipfelpunkte mittels eines lokalen Maximum-Filters. Für die Berechnung der Prominenz werden zwei alternative Algorithmen eingesetzt: der Bresenham-Algorithmus zur Pfadbestimmung in geradliniger Verbindung sowie der Dijkstra-Algorithmus zur Berechnung minimaler Pfade im Geländemodell. Die Dominanzbestimmung erfolgt über die Ermittlung des nächsthöheren Nachbarn und die Berechnung der horizontalen Distanz.

Alle Datenverarbeitungsschritte werden in Python realisiert, unter Einsatz etablierter Bibliotheken für numerische Berechnungen, Bildverarbeitung und Geodatenanalyse. Zur Interaktion mit dem Anwender wird eine grafische Benutzeroberfläche mit Visualisierung der DHM-Daten und der berechneten Gipfel implementiert. Die Validierung erfolgt durch Vergleich mit bestehenden Kartenwerken und durch Anwendung definierter Klassifikationskriterien, um die Korrektheit und Aussagekraft der Ergebnisse zu bewerten.

2 Theoretische Grundlagen

In diesem Kapitel werden grundlegende Theorien aus der aktuellen Forschung zur Förderung eines fundierten Verständnisses der Arbeit dargestellt. Da der Schwerpunkt der Arbeit von geowissenschaftlicher Natur ist, wird zunächst ein grober Überblick über die Geowissenschaften im Allgemeinen gegeben. Es folgt ein Abschnitt über die Modellierung von Höhendaten mittels digitaler Höhenmodelle. Abschließend werden für die Gipfelbestimmung relevante Parameter definiert und gängige Schwellwerte für die Gipfelklassifizierung erläutert.

2.1 Geowissenschaften

2.1.1 Überblick

Die Geowissenschaften, auch Erdwissenschaften genannt, bezeichnen die Gesamtheit der wissenschaftlichen Disziplinen, die sich mit dem Aufbau, der Zusammensetzung, der Dynamik und der Entwicklung der Erde beschäftigen. Der Begriff leitet sich vom altgriechischen Präfix „geo-" ab, was „Erde" bedeutet. [5] Im Falle der vorliegenden Arbeit erfolgt die Erfassung, Verarbeitung, Analyse und Visualisierung der Geodaten digital mittels informatischer Methoden. Man spricht hier von der Geoinformatik.

2.1.2 Disziplinen

Für die vorliegende Arbeit stehen vor allem jene geowissenschaftlichen Disziplinen im Vordergrund, die sich mit dem strukturellen Aufbau der Erdoberfläche auseinandersetzen. Sie gehören zu einem gemeinsamen Themenfeld, unterscheiden

sich jedoch hinsichtlich Fokus, Methodik und Zielsetzung. Alle geowissenschaftlichen Disziplinen prägen typischerweise entsprechende, geowissenschaftliche Adjektive (z.B. geodätisch oder topografisch), die in der vorliegenden Arbeit für die Umschreibung geowissenschaftlicher Sachverhalte verwendet werden. Um einen möglichst klaren Überblick über die Abgrenzungen zwischen den Begrifflichkeiten zu schaffen, werden die wichtigsten geowissenschaftlichen Disziplinen in den nachfolgenden Abschnitten beschrieben.

Geografie Die Geografie ist eine Geowissenschaft, die sich mit der räumlichen Struktur und Organisation der Erdoberfläche sowie den Wechselwirkungen zwischen Mensch und Umwelt befasst. Sie untersucht sowohl physische (z.B. Klima, Relief, Wasser) als auch anthropogene (z.B. Siedlungen, Wirtschaft) Strukturen. Im Unterschied zu anderen Geowissenschaften analysiert die Geografie Zusammenhänge und Prozesse, nicht nur Vermessungsdaten oder kartografische Darstellungen. [6]

Geodäsie Die Geodäsie ist eine Geowissenschaft, die sich mit der Erfassung, Vermessung und Abbildung der Erdoberfläche sowie ihrer Form, Schwereverteilung und zeitlichen Veränderungen befasst. Sie liefert präzise Koordinaten und georeferenzierte Daten als Grundlage für die Kartografie und die digitale Modellierung der Erdoberfläche. Dies geschieht vor allem über das sogenannte DHM, das im vorliegenden Projekt eine übergeordnete Rolle spielt und in Kapitel 2.2 näher erläutert wird. Im Unterschied zur Geografie analysiert die Geodäsie nicht die räumlichen Wechselwirkungen zwischen Mensch und Umwelt, sondern konzentriert sich auf die genaue geometrische Erfassung der Erde. [7]

Topografie Die Topografie ist ein Teilgebiet der Geodäsie, das sich mit der detaillierten Erfassung und Darstellung der natürlichen und künstlichen Formen der Erdoberfläche beschäftigt. Sie beschreibt Geländeformen, Höhenverhältnisse, Gewässer, Vegetation sowie Bauwerke meist in Form von Karten oder Plänen. Im Unterschied zur Geografie analysiert die Topografie keine übergeordneten räumlichen Zusammenhänge; im Gegensatz zur Geodäsie liegt der Fokus nicht auf der exakten Vermessung des Erdmodells, sondern auf der anschaulichen Wiedergabe

oberflächennaher Strukturen. Alltagssprachlich bezieht sich der Begriff „Topografie" oftmals auf die physische Beschaffenheit oder Geländeform einer Landschaft (z.B. „Die Topografie der Alpen"). [8]

Orografie Die Orografie ist eine geowissenschaftliche Disziplin, die sich mit der Lage, Form, Ausdehnung und Struktur von Gebirgen und Erhebungen auf der Erdoberfläche befasst. Sie beschreibt insbesondere die vertikale und horizontale Ausprägung der Erdoberfläche in gebirgigem Gelände. Zudem beschäftigt sie sich mit dem Einfluss von Bergen und Gebirgen auf atmosphärische, hydrologische und ökologische Prozesse, wie etwa Luftströmung, Niederschlag oder Erosion. Im Unterschied zur Topografie, die Geländeformen allgemein beschreibt, konzentriert sich die Orografie speziell auf gebirgige Strukturen und deren geowissenschaftliche Bedeutung. [9]

Kartografie Die Kartografie ist eine Geowissenschaft, die sich mit der Erstellung, Gestaltung und Nutzung von Karten zur Darstellung raumbezogener Informationen befasst. Sie übersetzt komplexe geografische, geodätische, topografische oder orografische Daten in eine verständliche visuelle Form. Dabei steht sie in enger Wechselwirkung mit anderen Geowissenschaften: Sie nutzt geodätische Messdaten zur präzisen Verortung, verarbeitet topografische Informationen zur Darstellung von Geländeformen und visualisiert geografische Analysen, etwa zu Klima, Landnutzung oder Bevölkerungsverteilung. Kartografie dient somit als zentrales Bindeglied zur Darstellung und Kommunikation geowissenschaftlicher Erkenntnisse. [10] Erfolgt die Kartografie im Rahmen der Geoinformatik digital, wird typischerweise ein sogenanntes DHM erstellt. Dieses Thema wird in Kapitel 2.2 vertieft.

2.1.3 Grundbegriffe

In der geowissenschaftlichen Literatur finden sich häufig fachspezifische Begriffe und Fachtermini. Einige dieser Begriffe leiten sich direkt aus geowissenschaftlichen Disziplinen wie der Geografie oder Topografie ab, wie in Kapitel 2.1.2 beschrieben. Daneben existieren weitere grundlegende Begriffe, die im geowissen-

schaftlichen Kontext allgemein verwendet werden. Die für diese Arbeit zentralen Begriffe werden in den folgenden Abschnitten kurz erläutert.

Relief Im geowissenschaftlichen Kontext bezeichnet das Relief die Form und Struktur der Erdoberfläche, einschließlich Höhenunterschieden und Gestaltmerkmalen wie Bergen, Tälern, Ebenen und Plateaus. [11] Darüber hinaus kann das Relief als „Rauheit" der Erdoberfläche beschrieben werden. [12]

Sattelpunkt Die Definition des Sattelpunkts kommt ursprünglich aus der Mathematik. Es handelt sich um eine spezielle Form des Wendepunkts mit Steigung 0. Wird der Sattelpunkt im dreidimensionalen Raum dargestellt, wird eine Fläche mit einem lokalen Minimum und einem lokalen Maximum modelliert. Die beiden Extrema der Fläche treffen sich dabei im Sattelpunkt. [13] Abbildung 2.1 verdeutlicht diesen mathematischen Sachverhalt:

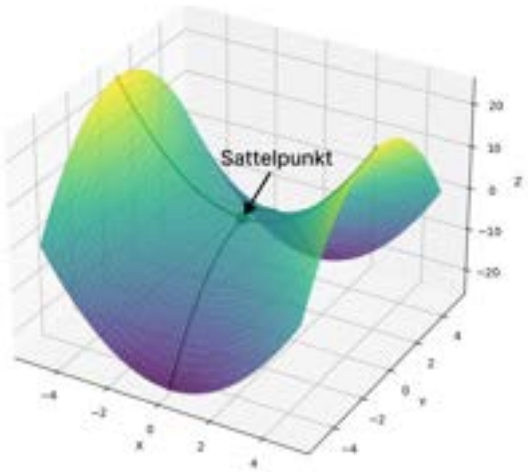

Abbildung 2.1: Geometrie eines Sattelpunkts

Es ist gut zu erkennen, dass der Sattelpunkt in der Mitte der modellierten Fläche keine Steigung besitzt. Er ist jedoch weder ein Minimum, noch ein Maximum, sondern eine Kombination der beiden Extrempunkte. Die mathematische Definition des Sattelpunkts kann analog für den geowissenschaftlichen Sattelpunkt angewandt werden.

Sattel Ein geografischer Sattel beschreibt der Bereich um einen Sattelpunkt. Es handelt sich um eine Geländeform in einem Gebirge, welche zwei Erhebungen miteinander verbindet. Im Vergleich zur Scharte ist der Übergang zwischen beiden Gipfeln dabei nicht spitz zulaufend, sondern sanft und langgestreckt. [14] Abbildung 2.1 verdeutlicht, warum der Bereich um den Sattelpunkt als Sattel bezeichnet wird: Die geometrische Struktur erinnert an einen Pferdesattel. Der Sattelpunkt spielt vor allem bei der Definition der Prominenz eine wichtige Rolle. Sie wird in Kapitel 2.3.2 definiert.

Scharte Auch als Joch bezeichnet, verbindet die Scharte ähnlich wie der Sattel zwei benachbarte Gipfel miteinander. Im Gegensatz zum Sattel ist die Scharte jedoch durch einen deutlich steileren bzw. schrofferen Einschnitt gekennzeichnet. In der Literatur werden die Begriffe Scharte und Sattel häufig synonym verwendet. [15]

2.2 Das digitale Höhenmodell

2.2.1 Definition

Das digitale Höhenmodell (DHM) ist ein Werkzeug der Geoinformatik zur digitalen Darstellung von Geländehöhen. In der Literatur ist oft auch der englische Begriff DEM zu finden; im weiteren Verlauf der Arbeit wird jedoch die deutsche Abkürzung DHM verwendet. [11] Es bildet die dreidimensionale Form der Erdoberfläche ab und dient der Analyse und Visualisierung topografischer Gegebenheiten. [16] Ein DHM ist „digital" im Sinne, dass es in elektronischem Format erzeugt, verteilt und analysiert wird. Es stellt die Höhe der Erdoberfläche in Form eines digitalen Bildes dar, bei dem jedes Pixel einen Höhenwert des Mittelpunkts des Pixels enthält. [11] Die Geodaten für die Modellierung stammen aus der Geodäsie und Topografie (siehe Abschnitte 2.1.2 und 2.1.2). Es ist zu beachten, dass es sich beim DHM um einen allgemeinen Oberbegriff handelt. Er umfasst spezifischere Modellformen, die in Abschnitt 2.2.2 erläutert werden.

2.2.2 Varianten

Als Oberbegriff umfasst das DHM spezifischere Modellformen wie das Digitales Geländemodell (DGM) und das Digitales Oberflächenmodell (DOM). Abbildung 2.2 stellt den Unterschied zwischen DGM und DOM grafisch dar:

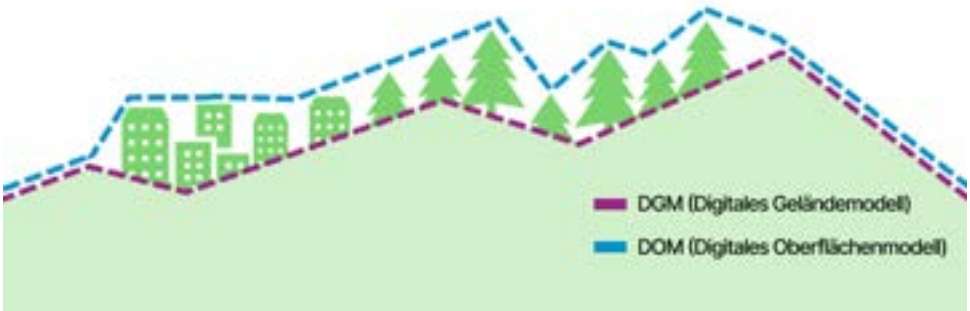

Abbildung 2.2: Unterschied zwischen DGM und DOM

Digitales Geländemodell (DGM) Ein DGM bezeichnet eine spezielle Form des DHM, bei der ausschließlich die unbewachsene Erdoberfläche abgebildet wird. In der Literatur ist oft auch der englische Begriff Digital Terrain Model (DTM) zu finden; im weiteren Verlauf der Arbeit wird jedoch die deutsche Abkürzung DGM verwendet. Der Begriff „Gelände" bezieht sich dabei explizit auf die natürliche Bodenoberfläche ohne Vegetation oder künstliche Objekte. DGMs stellen eine verfeinerte Version des DHMs dar, die durch zusätzliche Verarbeitungsschritte eine präzisere Erfassung markanter Geländestrukturen ermöglicht. [11]

Digitales Oberflächenmodell (DOM) Ein DOM repräsentiert die Höhen der obersten radarreflektierenden Oberfläche in einem bestimmten Gebiet. In der Literatur wird oft auch der englische Begriff Digital Surface Model (DSM) verwendet; im weiteren Verlauf der Arbeit wird weiterhin die deutsche Abkürzung DOM genutzt. Im Gegensatz zum DGM, welches ausschließlich die Geländeform ohne oberirdische Objekte abbildet, umfasst das DOM sämtliche, über dem Boden liegenden Strukturen wie Gebäude, Vegetation oder technische Anlagen. [11]

2.2.3 Erfassung der Höhendaten

Die Basis eines jeden DHMs bilden Geo- bzw. Höhendaten des zu untersuchenden Geländes, die systematisch erfasst, und zu einem digitalen Modell verarbeitet werden. [11] Der erste Schritt bei der Erstellung von DHMs bildet somit die Datenakquisition. Hierfür stehen mehrere Methoden zur Verfügung. In den folgenden Abschnitten werden die in der Literatur meistverwendeten Methoden beschrieben.

Bodenvermessung Bei der Bodenvermessung kommen traditionelle Techniken wie die Totalstation (TS) und Global Positioning System Real-Time Kinematic (GPS RTK) zum Einsatz [11] [17]. Sie liefern Koordinatenpunkte mit X-, Y- und Z-Werten (Höhe), welche die Grundlage für die spätere Modellierung bilden. Die Methode gilt als äußerst zuverlässig und genau, wobei insbesondere TS und GPS RTK geringere Fehler bei der Höhenmessung im Vergleich zu photogrammetrischen Ansätzen aufweisen können. Die Ergebnisse aus der Bodenvermessung

dienen deshalb oftmals als Referenz für die Bewertung der Genauigkeit anderer Methoden zur Datenerfassung. [17]

Light Detection and Ranging (LIDAR) Die LIDAR-Fernerkundung ist eine weit verbreitete, aktive Fernerkundungsmethode zur Erstellung von DHMs, bei der kurze Laserpulse ausgesendet und deren Rückkehrzeiten gemessen werden, um Entfernungen zu Objekten zu berechnen. Diese Methode generiert dichte 3D-Punktwolken mit X-, Y- und Z-Daten, die genaue Positions- und Höheninformationen der Oberfläche enthalten [18]. Ein Hauptvorteil ist die sehr hohe vertikale und horizontale Genauigkeit von oftmals wenigen Zentimetern. Zudem besitzt LIDAR die Fähigkeit zur Penetration dichter Vegetation, wodurch auch Bodenpunkte unterhalb des Bewuchses erfasst und präzise DGMs direkt gewonnen werden können. So kann LIDAR auch unter ungünstigen Bedingungen wie Regen oder Rauch operieren. Als Nachteil sind die hohen Kosten für die Datenerfassung und Verarbeitung anzuführen. Zudem erzeugt LIDAR sehr große Datenmengen und erfordert umfangreiche Rechenressourcen. [11]

Radio Detection and Ranging (RADAR) Die RADAR-Fernerkundung, insbesondere das Interferometric Synthetic Aperture Radar (IfSAR), ist eine weitere aktive Methode zur Erstellung von DHMs, die auf dem Senden und Empfangen von Mikrowellenpulsen basiert. IfSAR nutzt die Phaseninformationen von zwei leicht versetzten RADAR-Messungen, um ein Interferogramm zu erzeugen, aus dem Höhendaten abgeleitet werden. Ein wesentlicher Vorteil dieser Technologie ist ihre Effektivität unter ungünstigen Wetterbedingungen wie Nebel oder Bewölkung, sowie eine gewisse Vegetationspenetration, besonders bei längeren Wellenlängen. IfSAR eignet sich gut für die Abdeckung großer Flächen im regionalen bis kontinentalen Ausmaß [18]. Es bietet jedoch mit einigen Metern eine geringere räumliche Auflösung und Genauigkeit als LIDAR. Zudem ist es aufgrund von Schatteneffekten weniger geeignet für steiles Gelände oder hohe städtische Strukturen. [11]

Photogrammetrie Die Photogrammetrie nutzt das Prinzip der Stereobildgebung. Hierbei wird dasselbe Objekt von mindestens zwei leicht versetzten Kamerapositionen aufgenommen, um Höheninformationen abzuleiten und ein 3D-Stereomodell

der Oberfläche zu erstellen. Diese Methode bietet den Vorteil eines reichen fotografischen Kontextes der Landschaft, was die Interpretation und Klassifizierung von Merkmalen erleichtert, und kann eine kostengünstige Option darstellen, insbesondere wenn bereits vorhandene Stereoaufnahmen genutzt werden können. Als passive Fernerkundungstechnologie bildet die Photogrammetrie jedoch nur die oberste sichtbare Oberfläche ab. Sie ist daher wetterabhängig und kann keine Messungen durch dichte Vegetationsbedeckung erfassen. Darüber hinaus weist sie eine geringere räumliche Auflösung im Vergleich zu aktiven Systemen auf. [11] Automatisierte photogrammetrische Systeme können systematische Fehler erzeugen und erfordern oft eine manuelle Nachbearbeitung durch erfahrene Bediener, um die Datenqualität zu gewährleisten. [18]

2.2.4 Modellierung der Höhendaten

Durch die in Kapitel 2.2.3 gezeigten Methoden zur Geodatenerfassung werden die Höhendaten erfasst. Je nach eingesetzter Erfassungsmethode handelt sich dabei typischerweise um einzelne, unregelmäßig im Raum verteilte Punkte. Jeder dieser Punkte beinhaltet eine X-, Y- und Z-Koordinate mit der entsprechenden Höheninformation. Die Gesamtheit der Punkte wird auch als dreidimensionale Punktwolke bezeichnet. Für diese dreidimensionalen Punktwolkendaten existieren dedizierte Datenformate wie das XYZ-Format mit der Endung „.xyz" oder das LAS-Format mit der Endung „.las". Diese Rohdaten bilden die Basis für den nachfolgend beschriebenen Modellierungsprozess des DHMs. [19]

Im ersten Schritt der Vorverarbeitung werden die Punktwolkendaten gefiltert bzw. klassifiziert, um Störfaktoren wie Vegetation, Gebäude oder Messrauschen zu identifizieren. Ein zentraler Bestandteil ist dabei die Klassifikation der Punkte, insbesondere die Trennung zwischen Boden- und Objektpunkten. [11] Soll als Zielmodell ein DGM erstellt werden, sind nur die Bodenpunkte relevant. Soll hingegen ein DOM als Zielmodell modelliert werden, müssen auch Vegetations- und Gebäudestrukturen berücksichtigt werden. Näheres zu den Varianten von DHMs ist in Kapitel 2.2.2 beschrieben. Als Ergebnis entstehen Vektordaten. Sie stellen die Objekte der realen Welt als raumbezogene Geometrien dar. Zu den geometrischen Strukturen gehören Punkte, Linien und Polygone. Obwohl sie topografische Merkmale wie Höhenlinien speichern können, sind sie für die Darstellung der gesamten

Geländehöhe noch ungeeignet. Auch für Vektordaten gibt es dedizierte Datenformate wie das Shapefile-Format mit der Endung „.shp" oder das MIF/MID-Format mit den Endungen „.mif" und „.mid". [19]

Anschließend erfolgt die Modellierung der Geländeoberfläche. Hierfür werden die Vektordaten in ein Triangulated Irregular Network (TIN) überführt. [17] Bei einem TIN wird die Oberfläche als eine Reihe von zusammenhängenden, nicht überlappenden Dreiecken dargestellt. Die Eckpunkte der Dreiecke bestehen aus den unregelmäßig verteilten Messpunkten der Punktwolke und beinhalten die Höheninformationen. [16] Die TIN-basierte Modellierung erlaubt eine flexible und präzise Beschreibung des Geländes, insbesondere bei inhomogen verteilten Punktdaten. So eignet sie sich vor allem für die Modellierung komplexer oder filigraner Geländestrukturen, wie beispielsweise Gräben oder Bachläufe. [11] Alternativ können direkte Interpolationsverfahren wie Inverse Distance Weighting (IDW), Kriging oder Spline-Methoden verwendet werden. Hierbei wird aus den diskreten Punkten eine kontinuierliche Höhenfläche erzeugt.

Das Ergebnis der Modellierung wird anschließend in ein Rasterformat überführt. Hierbei wird die Oberfläche in ein Gitter aus regelmäßigen Zellen bzw. Pixeln unterteilt. [11] Jeder Rasterzelle wird ein mittlerer, minimaler, maximaler oder interpolierter Höhenwert zugewiesen. [19] Die erzeugten Rasterdaten erlauben später die einfache Anwendung zellenbasierter Algorithmen für verschiedene topografische Berechnungen.

Das resultierende Rastermodell stellt das eigentliche digitale Höhenmodell (DHM) dar. Es kann in der Praxis über verschiedene Datenformate wie beispielsweise das ASCII Grid-Format mit der Endung „.asc" oder das USGS DEM-Format mit der Endung „.dem" ausgegeben werden. Das gängigste Datenformat für Rasterdaten bzw. DHMs ist jedoch das GeoTIF-Format mit der Endung „.tif", auf das in Kapitel 2.2.6 näher eingegangen wird. Es handelt sich hierbei um den Industriestandard, der eine breite Softwareunterstützung und gute Komprimierung bietet. [11] Die fertige DHM-Datei kann nun für weitere Analysen, wie beispielsweise Hangneigungs-, Einzugsgebiets- oder Sichtbarkeitsanalysen in ein gängiges Geoinformationssystem (GIS) importiert werden. [19]

2.2.5 Einflussfaktoren auf die Genauigkeit

Die Genauigkeit, insbesondere in vertikaler Richtung, ist das primäre Qualitätsmerkmal von DHMs. Sie beschreibt die Übereinstimmung eines geschätzten Höhenwertes mit dem tatsächlichen. [11] Die Literatur geht im Allgemeinen von vier hauptsächlichen Einflussfaktoren auf die Genauigkeit von DHMs aus, welche in den folgenden Abschnitten gezeigt werden.

Genauigkeit der Quelldaten Die Genauigkeit der Quelldaten ist ein entscheidender Einflussfaktor für die finale Genauigkeit des DHMs. [18] Sie ist vor allem von der gewählten Methode zur Geodatenerfassung abhängig, welche in Kapitel 2.2.3 näher beschrieben sind. [11] So zeigt die vergleichende Studie von Roblero-Hidalgo et al. (2020), dass DHMs, die mit der klassischen Bodenvermessung generiert wurden, kleinere absolute Fehler in den Z-Koordinaten aufweisen, als die mit LIDAR- oder RADAR-Sensoren erzeugten DHMs. [17]

Auflösung der Quelldaten Je nachdem, wie das Abtastintervall beim Erfassen der Geodatenpunkte gewählt wird, ändert sich die Auflösung bzw. Dichte der Quelldaten. Sie hat einen direkten Einfluss auf die Genauigkeit des DHMs: Nimmt die Auflösung der Quelldaten zu, so steigt auch die Genauigkeit des DHMs. [18] Generell ist eine höhere Auflösung vor allem in Gebieten mit schnellen Höhenänderungen, wie in steilem Gelände oder in städtischen Gebieten, erforderlich. [11] Die Auflösung ist hierbei wieder von den Datenerfassungsmethoden abhängig, die in Kapitel 2.2.3 beschrieben werden. Es besteht ein Kompromiss zwischen der gewünschten Auflösung und der Datenhandhabung (Speicherung, Verwaltung und Analyse), da eine hohe Auflösung oftmals zu deutlich größeren Datenmengen und höheren Produktionskosten führen. [11] [16]

Topografie der untersuchten Geländeoberfläche Die Beschaffenheit der Geländeoberfläche, insbesondere das Relief, sind ein wichtiger Faktor für die Genauigkeit von DHMs. Allgemein lässt sich feststellen, dass die Genauigkeit mit zunehmendem Relief abnimmt. Es gibt jedoch eine interessante Ausnahme: Die Tests in der Studie von Gong et al. (2000) zeigen, dass die höchste Genauigkeit in hügeligen Gebieten erzielt werden kann. In sehr flachem Gelände ist die Genauigkeit

hingegen geringer. [18] Die Komplexität des Geländes kann auch die Wahl der Erfassungsmethode beeinflussen. So können bei der Erfassung mit LIDAR- oder RADAR-Sensoren in steilem Gelände oder in städtische Gebieten durch Abschattungen Lücken entstehen. [11]

Wahl des Modellierungsansatzes Die Wahl des Modellierungsansatzes für die Erstellung des DHMs hat einen signifikanten Einfluss auf die Genauigkeit. Dabei kann in direkte und indirekte Modellierungsansätze unterschieden werden. [18] Ein Beispiel für einen direkten Modellierungsansatz ist das TIN. Eine indirekte Modellierungsmethode ist beispielsweise die Zufalls-zu-Gitter-Interpolation zur Bildung eines Gitter-Netzwerks. [11] Nach Gong et al. (2000) führt der direkte Modellierungsansatz zu deutlich genaueren Ergebnissen im Vergleich zum indirekten Modellierungsansatz. [18] Weitere Informationen zu den Modellierungsmethoden von DHMs sind in Kapitel 2.2.4 beschrieben.

2.2.6 GeoTIFF

Wie in Kapitel 2.2.4 beschrieben, hat sich das GeoTIF-Format als Standard für die Speicherung und den Austausch von Rasterdaten bzw. DHMs etabliert. Folglich sind die meisten öffentlich zugänglichen DHMs nur als GeoTIFF-Dateien verfügbar, weshalb diesem Dateiformat in dieser Arbeit eine besondere Bedeutung zukommt. Es handelt sich dabei um eine Erweiterung des Tagged Image File Format (TIFF) und besitzt ebenfalls die Dateiendung „.tif". [20] Das TIFF ist ein Dateiformat für die verlustfreie Speicherung von Bilddaten im Rasterformat. Es eignet sich für die Speicherung der Rasterdaten von DHMs, da hier eine hohe Abbildungsgenauigkeit benötigt wird. [21]

Neben den Höhendaten im Rasterformat speichert eine GeoTIFF-Datei zusätzliche Informationen zur Georeferenzierung. [21] Diese Parameter werden als Metadaten mithilfe sogenannter Tags direkt in die TIFF-Datei eingebettet und dienen der räumlichen Zuordnung der Rasterdaten auf die reale Erdoberfläche. [20] Im Detail handelt es sich dabei um folgende Zusatzparameter:

Die **geometrischen Referenzpunkte** beinhalten die geografischen Koordinaten der oberen linken Ecke des Rasterbildes. Mithilfe dieser Referenzpunkte können

die Pixelkoordinaten exakt auf die entsprechenden geografischen Koordinaten abgebildet und das Bild in der realen Welt verortet werden. [20] [22]

Das **Coordinate Reference System (CRS)** gibt die Art (geografisch oder projiziert) und die Bezeichnung (European Petroleum Survey Group Geodesy (EPSG)-Code) des verwendeten Koordinatenreferenzsystems an. Ein typisches Beispiel für ein geografisches Koordinatensystem ist das weit verbreitete WGS84-System mit seinen Längen- und Breitengraden. Ein Beispiel für das projektierte Koordinatensystem ist das UTM-System mit seiner Zoneneinteilung. [19] [22]

Die **räumliche Auflösung** gibt die physische Größe eines einzelnen Pixels in der realen Welt an. Dieser Wert wird typischerweise in der Einheit des zugrundeliegenden CRS, beispielsweise in Meter oder Grad pro Pixel, angegeben. [20] [22]

Die **Transformationsmatrix** ermöglicht die präzise Konvertierung von den Pixelkoordinaten zu den geografischen Koordinaten. [20]. Sie ist besonders wichtig, falls das im GeoTIFF enthaltene Bild gedreht oder verzerrt ist. [22]

Handelt es sich um ein projiziertes CRS, wird zusätzlich ein **Ellipsoid-Modell** zur Approximation der Erdform angegeben. Es beschreibt, wie die gekrümmte Erdoberfläche auf eine flache Ebene abgebildet bzw. projiziert wird. [20] [22]

Abbildung 2.3 zeigt beispielhaft den Aufbau einer GeoTIFF-Datei:

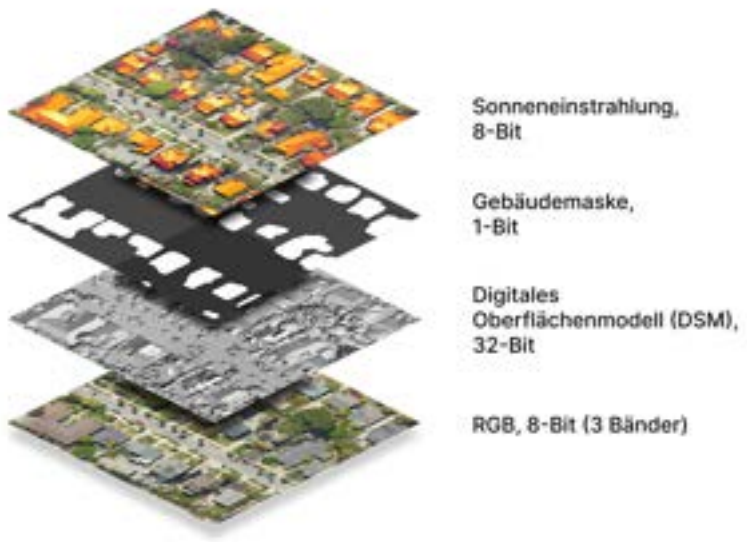

Abbildung 2.3: Beispiel für die Multiband-Unterstützung bei GeoTIFF [23]

In der Abbildung ist zu erkennen, dass eine TIFF bzw. GeoTIFF-Datei mehrere Rasterbilder enthalten kann, was als Multiband-Unterstützung bezeichnet wird [22]. Dabei werden mehrere Datenbänder in einer Datei gespeichert, was vor allem für Multispektralbilder oder DHMs von Bedeutung ist. Bei letzteren ist es allgemein üblich, dass das DHM in einem der ersten Bänder abgespeichert ist. Darüber hinaus werden weitere Bänder für die Speicherung zusätzlicher Informationen, wie Vegetation oder Temperatur genutzt. [21] Im Falle der Abbildung 2.3 sind vier unterschiedliche Rasterbilder desselben Geländes in einer GeoTIFF-Datei gespeichert. Neben dem DHM werden zusätzlich die sichtbaren Rot Grün Blau (RGB)-Farbwerte, eine binäre Gebäudemaske und Messwerte der Sonneneinstrahlung auf den Gebäudedächern abgespeichert. [23]

Zu beachten ist, dass alle Bänder innerhalb der TIFF- bzw. GeoTIFF-Datei unterschiedliche Bittiefen besitzen können. Die Bittiefe gibt an, mit wie vielen Bits das Farbschema eines Bildes kodiert ist. Je komplexer die darzustellenden Farbabstufungen sind, desto größer muss die Bittiefe sein. Beispielsweise werden Graustufenbilder oder Bilder mit indexierten Farben mit einer Bittiefe von 8 Bit kodiert. Bei DHMs werden häufig Bittiefen von 16 oder 32 Bit verwendet, da hier eine deutlich höhere Präzision gefordert ist. [24] [22]

2.3 Kriterien zur Klassifizierung von Gipfeln

Um Gipfel absolut identifizieren und vergleichen zu können, bedarf es objektiver Klassifizierungsmethoden. Diese beruhen meist auf wenigen Kriterien, die in den folgenden Kapiteln definiert werden.

2.3.1 Höhe

Der Begriff der Höhe scheint auf den ersten Blick banal. Jedoch ergeben sich je nach betrachtetem Referenzniveau unterschiedliche Definitionen. Allgemein ist zu sagen, dass die Höhe dem positiven Abstand eines Punktes von einer gewählten Referenzfläche entspricht. Beim DHM ist ausschließlich die Höhe der Erdoberfläche gesucht, weshalb sich das Problem auf den Abstand zwischen topografischer Oberfläche und gewählter Referenzfläche reduziert. Die topografische Oberfläche ist eine mathematisch vereinfachte und maßstabsabhängige Repräsentation der Erdoberfläche. Sie ist einwertig, was bedeutet, dass sie keine Höhlen oder überhängende Klippen besitzt. [16] Abbildung 2.4 zeigt drei verschiedene Definitionen der Höhe mit ihren zugehörigen Referenzflächen und der topografischen Oberfläche:

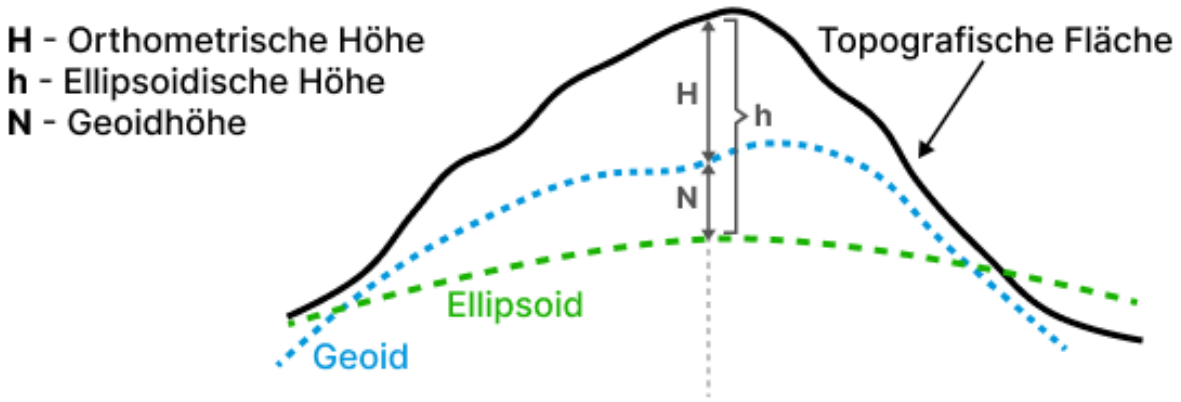

Abbildung 2.4: Höhendefinitionen im Überblick [16]

Orthometrische Höhe (H) Die verbreitetste und für die vorliegende Arbeit relevante Höhendefinition ist die „Höhe über dem Meeresspiegel" [4]. Sie wird auch

als orthometrische Höhe bezeichnet und nutzt den mittleren Meeresspiegel, auch Geoid genannt, als Referenzniveau. Das Geoid ist ein Modell der Äquipotenzialfläche des Erdschwerefeldes, die am mittleren Meeresspiegel optimal anschließt und sich unter den Kontinenten fortsetzt. Es ist die physische Form der Erde, die durch die Verteilung der Massen im Erdinneren und die Rotation der Erde bestimmt wird. Das Geoid ist im Gegensatz zum Ellipsoid eine unregelmäßige, aber physikalisch definierte Fläche. [16]

Ellipsoidische Höhe (h) Sie gibt den positiven Abstand zum Ellipsoid als Referenzniveau an. Es handelt sich dabei um ein mathematisch definiertes Rotationsellipsoid, das als Näherung für die Erdform dient. Es ist eine glatte, geschlossene Oberfläche, die für geodätische Berechnungen verwendet wird. [16]

Geoidhöhe (N) Zuletzt kann mit den zuvor definierten Referenzflächen eine weitere Höhe definiert werden: Die Geoidhöhe repräsentiert die Höhendifferenz zwischen Ellipsoid und Geoid. Dieser Wert kann positiv oder negativ sein und eine Größenordnung von bis zu 100 m aufweisen. [16]

2.3.2 Prominenz

Die Prominenz ist ein Maß für die Eigenständigkeit eines Berges oder Gipfels. Sie beschreibt den Höhenunterschied zwischen dem Gipfel und dem höchsten Punkt, zu dem man absteigen muss, bevor man einen höheren Gipfel erreichen kann. Dieser Punkt wird als Scharte bezeichnet, weshalb die Prominenz auch Schartenhöhe genannt wird. [4] Eine ausführlichere Definition der Scharte ist in Kapitel 2.1.3 gegeben. Abbildung 2.5 stellt die Prominenz mit der zugehörigen Scharte grafisch dar:

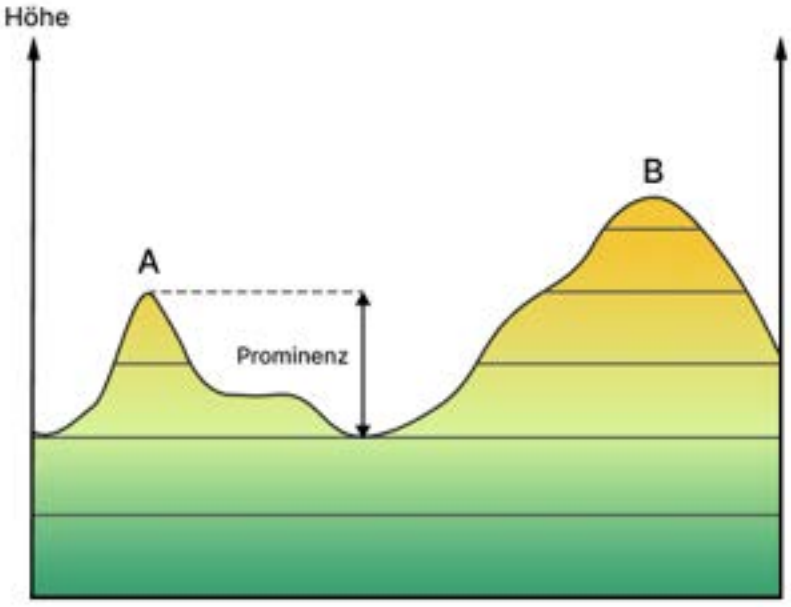

Abbildung 2.5: Darstellung der Prominenz

Die Abbildung zeigt ein zweidimensionales Schnittbild der Erdoberfläche mit den beiden Gipfeln A und B. Die Scharte ist hier der höchste Tiefpunkt, der durchschritten werden muss, um von Gipfel A zu Gipfel B zu gelangen. Es ist zu beachten, dass dieser Punkt nicht immer auf der geraden Luftlinie zwischen den beiden Gipfeln liegen muss, wie die Abbildung 2.5 suggeriert. Die Prominenz von Gipfel A berechnet sich anschließend aus der Differenz seiner Höhe und der Höhe der Scharte. [4]

Die Prominenz erlaubt eine objektive Einordnung der Bedeutung eines Berges in seinem topografischen Umfeld. Je größer die Prominenz, desto isolierter und dominanter erscheint der Berg in der Landschaft. In den Geowissenschaften dient die Prominenz als wichtiges Kriterium zur Klassifikation von Gebirgserhebungen. Sie ermöglicht es, Gipfel mit unterschiedlichen Höhenlagen miteinander zu vergleichen. [4]

2.3.3 Dominanz

Die Dominanz ist ein Maß für die topografische Isolation eines Gipfels. Sie gibt an, inwieweit er seine Umgebung überragt bzw. dominiert. Es geht dabei um die horizontale Entfernung, zwischen einem Gipfel und einem nächsthöheren Punkt. Es handelt sich folglich um den maximalen Radius um den Gipfel, innerhalb dessen kein anderer, höherer Punkt existiert. [4] Je größer dieser Radius ist, desto isolierter ist der Gipfel von anderen Erhebungen [25]. Die Definition impliziert, dass der Gipfel des Mount Everest, als höchste Erhebung der Erde, eine unendlich große Dominanz besitzt. Abbildung 2.6 illustriert die Dominanz eines Gipfels grafisch:

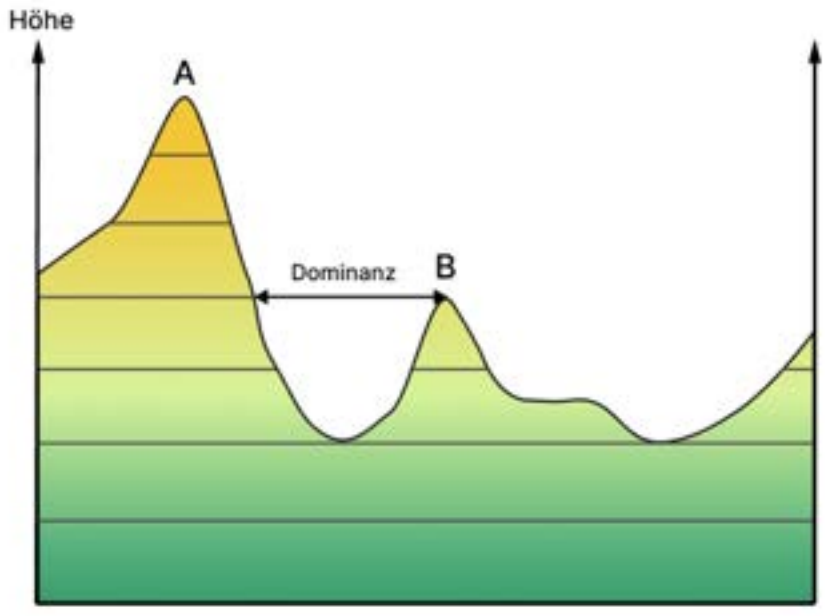

Abbildung 2.6: Darstellung der Dominanz

In der Abbildung ist erneut ein zweidimensionales Schnittbild der Erdoberfläche mit den Gipfeln A und B dargestellt. Es ist gut erkennen, dass es sich bei der Dominanz von Gipfel B um die Distanz vom Gipfelpunkt bis zum nächstgelegenen Punkt auf gleicher Höhe dargestellt. [4]

Die Dominanz liefert einen objektiven Vergleichswert der topografischen Ausprägung eines Gipfels. In den Geowissenschaften wird sie häufig zur Klassifikation und Bewertung von Gipfeln verwendet. Sie erlaubt Vergleiche zwischen Gipfeln hinsichtlich ihrer relativen Isolation. [4] [25]

2.3.4 Orografische Dominanz

Die orografische Dominanz ist ein weiteres Maß zur objektiven Klassifizierung von Gipfeln und Bergen und wurde von Eberhard Jurgalski entwickelt. Da sie den relativen Höhenunterschied zwischen der Höhe und der Prominenz eines Gipfels beschreibt, wird sie auch „relative Prominenz" genannt. Die orografische Dominanz (oD) wird in Prozent angegeben und kann mithilfe der folgenden Formel berechnet werden:

$$oD = \frac{P}{H} \cdot 100$$

Das P steht dabei für die Prominenz eines Gipfels. H gibt die orthometrische Höhe des Gipfels an. [4] Trotz der ähnlichen Bezeichnung steht die orografische Dominanz nicht in direktem Zusammenhang mit der allgemeinen Dominanz, die in Kapitel 2.3.3 definiert wird. Sie findet jedoch, ähnlich wie die allgemeine Dominanz, in vielen Systemen zur Gipfelklassifizierung Anwendung. Weitere Informationen zu Systemen und Schwellwerten für die Gipfelklassifizierung werden in Kapitel 2.4 gegeben.

2.4 Schwellwerte zur Klassifizierung von Gipfeln

In der geowissenschaftlichen Fachliteratur existiert keine universell gültige Definition von Gipfeln. So meint auch Jurgalski (2015): „In der internationalen Diskussion gibt es keine klare Definition von einem „Berg" oder „Gipfel"". [4] Aus diesem Grund befinden sich verschiedene Klassifizierungssysteme und -methoden eigenständiger Institutionen im Umlauf. Sie definieren Schwellwerte für universell definierte Kriterien bzw. Parameter, um Gipfel miteinander vergleichen und klassifizieren zu können. Die Relevantesten dieser Kriterien werden in Kapitel 2.3 definiert. In den folgenden Kapiteln werden einige dieser Klassifizierungsmethoden und ihre zugehörigen Schwellwerte für die Klassifizierungskriterien vorgestellt.

2.4.1 UIAA

Beim UIAA handelt es sich um den internationalen Dachverband der Alpinismusverbände. Er liefert für die Alpen eine recht einfache Definition für die Bestimmung eines Gipfels. Sie beruht auf einem einzelnen Kriterium: Besitzt eine Erhebung in den Alpen eine Prominenz von mindestens 30 m, so handelt es sich um einen eigenständigen Gipfel. Dieses Kriterium bildet die Grundlage für die offizielle Liste der 4000er-Gipfel der Alpen. Diese Liste führt alle Erhebungen in den Alpen, die eine Mindesthöhe von 4000 m besitzen und das genannte Kriterium erfüllen. [25]

2.4.2 Das erweiterte Prominenzkonzept

Das von Eberhard Jurgalski entwickelte erweiterte Prominenzkonzept nutzt die orografische Dominanz als einzelnes Kriterium zur Klassifizierung von Gipfeln. Die orografische Dominanz ist in Kapitel 2.3.4 definiert. Im Unterschied zum UIAA-System teilt diese Methode die Erhebungen nicht nur in Gipfel und Nicht-Gipfel ein; sie definiert verschiedene Gipfelklassen mit unterschiedlichen Schwellwerten, wie beispielsweise Haupt- oder Nebengipfel. [4] Die gesamte Einteilung der Gipfelklassen inklusive Schwellwerte ist in Tabelle 2.1 dargestellt:

Tabelle 2.1: Schwellwerte des erweiterten Prominenzkonzepts[4] [25]

Gipfelklasse	***oD***	**Beispiel**
Weltberg	10,5%	Mont Blanc (97,7%)
Berg	7,0%	Schwarzenstein (10,2%)
Hauptgipfel erster Ordnung	4,2%	Wilder Pfaff (3,6%)
Hauptgipfel zweiter Ordnung	2,1%	
Nebengipfel erster Ordnung	1,4%	Elwertätsch (1,2%)
Nebengipfel zweiter Ordnung	0,7%	

Die Tabelle zeigt: Für Jurgalski gilt eine Erhebung mit einer orografischen Dominanz von mindestens 2,1 % als Hauptgipfel. Liegt die orografische Dominanz über einem Wert von 7 %, spricht er von einem Berg. Jede der drei Hauptkategorien (Berg, Hauptgipfel und Nebengipfel) ist wiederum in zwei Unterkategorien unterteilt, sodass insgesamt sechs Klassen entstehen. In der rechten Spalte der Tabelle sind reale Gipfel als Beispiel für die einzelnen Klassen aufgeführt.

2.4.3 Eigenständigkeitsklassen

Das Klassifizierungssystem der Eigenständigkeitsklassen wurde von Wolfgang Leonhard entwickelt. Statt die einzelnen Parameter eines Gipfels separat zu bewerten, kombiniert es mehrere Kriterien zu einem Gesamtwert. Dieser wird als Eigenständigkeit E bezeichnet. Für ihre Berechnung werden drei Kriterien berücksichtigt. Diese werden mit den folgenden Formeln in einen Wertebereich von 0 bis 7 überführt:

- Höhe: $E_H = -log_2(Gipfelhhe/8848)$
- Dominanz: $E_D = -log_2(Dominanz/100000)$
- orografische Dominanz bzw. relative Prominenz: $E_p = -log_2(oD/100)$

[25] Die Tabelle 2.2 zeigt, welche Kriterien erforderlich sind, um einen bestimmten Wert zu erreichen.

Tabelle 2.2: Definition der Indizes der Eigenständigkeit [25]

Index	**Höhe (E_H)**	**Dominanz (E_D)**	***oD* (E_p)**
0	8848m	100000m	100,0%
1	4424m	50000m	50,0%
2	2212m	25000m	25,0%
3	1106m	12500m	12,5%
4	553m	6250m	6,3%
5	276,5m	3125m	3,1%
6	138,3m	1562,5m	1,6%
7	69,1m	781,3m	0,8%

In der Tabelle entspricht beispielsweise $E_H = 0$ einer Höhe von 8848 m, was dem höchsten Punkt der Erde entspricht. Aus der Einteilung wird ersichtlich: Je niedriger der Wert eines Index ist, desto höher ist die Eigenständigkeit des Gipfels. Der Mittelwert der drei Einzelindizes E_H, E_D und E_p ergibt die Gesamtbewertung E, anhand derer acht Eigenständigkeitsklassen unterschieden werden. Diese Klassen entsprechen verschiedenen Gipfel- bzw. Bergtypen. So steht beispielsweise eine Eigenständigkeitsklasse von 0 für einen Weltberg, während die Klasse 5 einem Nebengipfel entspricht.

3 Stand der Technik

Dieses Kapitel baut auf den theoretischen Grundlagen der vorangegangenen Abschnitte auf und untersucht den aktuellen Stand der Technik im Bereich der automatisierten Gipfelerkennung. Dazu werden verschiedene Ansätze zur Gipfelerkennung vorgestellt. Anschließend folgt eine detaillierte Darstellung der Algorithmen zur Erkennung potenzieller Gipfel sowie zur Berechnung der Prominenz und Dominanz.

3.1 Ansätze zur Gipfelfindung

Grundsätzlich existieren verschiedene Methoden zur automatisierten Gipfelerkennung. In dieser Arbeit werden drei Ansätze exemplarisch untersucht und miteinander verglichen: die Filterung anhand von Prominenz und Dominanz, die heuristische Filterung basierend auf der lokalen Umgebung sowie die multiskalige Fuzzy-Klassifizierung.

3.1.1 Filterung nach Prominenz und Dominanz

Dieser Ansatz basiert auf der Analyse charakteristischer Merkmale von Gipfeln. Zunächst werden potenzielle Kandidaten identifiziert, anschließend erfolgt die Berechnung von Prominenz und Dominanz für jeden dieser Punkte. Übersteigen die ermittelten Werte definierte Schwellen, werden die Punkte als Gipfel eingestuft. Dieses Verfahren wurde bereits in früheren Projekten eingesetzt [1], [2].

Nach Torres et al. erzielt dieser Ansatz unter den drei betrachteten Methoden den geringsten durchschnittlichen Distanzfehler, zeigt jedoch eine erhöhte Anzahl

falsch positiver Ergebnisse [26]. Ein zentraler Vorteil liegt in der guten Steuerbarkeit, da die Resultate direkt über die gewählten Schwellenwerte beeinflusst werden können. Zudem sind die erzeugten Gipfeldefinitionen gut vergleichbar, da viele etablierte Kriterien auf Prominenz und Dominanz basieren (siehe Kapitel 2.4).

3.1.2 Heuristische Filterung nach lokaler Umgebung

Dieser Ansatz basiert auf der Annahme, dass ein Gipfel ein Punkt ist, der von einer definierten Anzahl niedrigerer Punkte umgeben ist. Zunächst werden potenzielle Gipfelkandidaten ermittelt, die eine festgelegte Mindesthöhe überschreiten. Für jeden dieser Punkte wird anschließend geprüft, ob das umliegende Gelände einen bestimmten Höhenunterschied unterschreitet. Erfüllt ein Punkt beide Bedingungen, wird er als Gipfel klassifiziert. Das Ergebnis ist eine Rasterkarte, die zwischen Gipfelpunkten, deren räumlicher Ausdehnung und Nicht-Gipfelpunkten unterscheidet [26].

Diese Methode weist im Vergleich zum Prominenz-Dominanz-Ansatz einen größeren mittleren Distanzfehler auf. Aber es treten weniger falsch positive Ergebnisse auf und diese befinden sich auch überwiegend in Gebieten, in denen Gipfel typischerweise erwartet werden [26].

3.1.3 Multiskalige Fuzzy-Klassifizierung

Bei diesem Ansatz zur Gipfelidentifikation wird eine multiskalige Analyse durchgeführt, bei der das Gelände aus verschiedenen Betrachtungsebenen bewertet wird. Die Untersuchung umfasst sowohl die lokale Hangneigung als auch die Veränderung der Neigung über größere Distanzen. Die resultierenden Werte repräsentieren eine quantitative Bewertung der „Gipfelhaftigkeit" jedes Punktes des Höhenmodells. Diese Bewertung wird in einem Datensatz abgebildet. Für die exakten Gipfelkoordinaten ist eine Nachbearbeitung erforderlich. [26]

Nach Torres et al. zeigt auch dieser Ansatz eine Tendenz zu falsch positiven Ergebnissen in typischen Gipfelregionen. Jedoch ist die Methode in der praktischen

Umsetzung anspruchsvoll, da sie eine geeignete Heuristik sowie eine komplexe Nachbearbeitung zur präzisen Identifikation der tatsächlichen Gipfelpunkte erfordert [26].

Für die vorliegende Arbeit wird der Ansatz der Filterung nach Prominenz und Dominanz verwendet, da dieser den geringsten Distanzfehler aufweist und eine gute Kontrollierbarkeit bietet.

3.2 Bestimmung potenzieller Gipfel

Als erster Schritt für den Ansatz der Prominenz- und Dominanz-Filterung ist die Identifizierung potenzieller Gipfelkandidaten notwendig. Hierfür kommen zwei Ansätze infrage: die morphologischen Operationen und der Maximum-Filter. Im Folgenden werden diese beiden Algorithmen beschrieben und anschließend hinsichtlich ihrer Eignung für die Bestimmung potenzieller Gipfel verglichen.

3.2.1 Morphologische Operationen

Morphologische Operationen sind Verfahren der digitalen Bildverarbeitung, die auf der Mengenlehre basieren. Sie verändern die Struktur eines Bildes, indem ein sogenanntes strukturierendes Element schrittweise über das Bild bewegt und dabei die Bildinformationen angepasst werden. Zu den grundlegenden Operationen zählen Erosion und Dilatation. [27]

Erosion Die Erosion ist eine morphologische Operation, die zur Verkleinerung von Objekten im Vordergrund eines Bildes führt. Dabei wird für jeden Bildpunkt geprüft, ob das strukturierende Element vollständig innerhalb des Objekts liegt, also vollständig von Vordergrundpixeln überdeckt wird. Nur wenn diese Bedingung erfüllt ist, bleibt der entsprechende Punkt im Ergebnisbild erhalten. Anschaulich bedeutet dies, dass die Ränder eines Objekts nach innen „erodiert" werden. Dadurch werden schmale Verbindungen oder kleine Strukturen zwischen Objekten entfernt [27], wie Abbildung 3.1 zeigt:

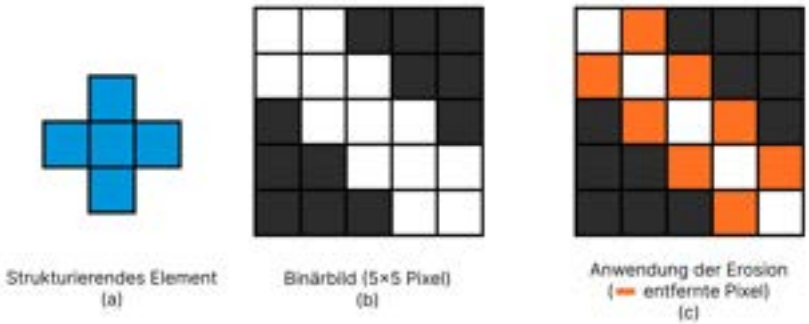

Abbildung 3.1: Beispielsergebnis der Anwendung der Erosion

In (a) ist das verwendete strukturierende Element dargestellt, das Form und Größe des Prüfbereichs definiert. Bild (b) zeigt das Eingabebild mit einer dicken Linie. Nach Anwendung der Erosion ist in (c) zu sehen, dass die Linie im Ergebnisbild deutlich schmaler erscheint.

Dilatation Die Dilatation ist das Gegenstück zur Erosion und führt zu einer Vergrößerung oder Verdickung von Objekten im Bild. Dabei wird das strukturierende Element über das Bild verschoben. Ein Bildpunkt wird im Ergebnisbild dem Vordergrund zugeordnet, wenn das zentrierte strukturierende Element an dieser Position mindestens einen Vordergrundpixel des Originals überdeckt. Dadurch wachsen die Objektgrenzen nach außen. Umfang und Form der Ausdehnung hängen unmittelbar von Größe und Geometrie des verwendeten strukturierenden Elements ab. [27] Abbildung 3.2 zeigt eine Anwendung der Dilatation auf ein binäres Bild.

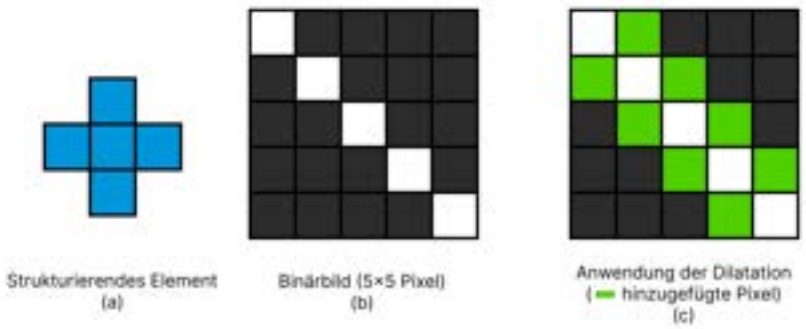

Abbildung 3.2: Beispielsergebnis der Anwendung der Dilatation

Abbildung (a) zeigt das verwendete strukturierende Element. Das Eingabebild ist in (b) dargestellt und enthält eine dünne Linie. Nach Anwendung der Dilatation ist in (c) zu erkennen, dass die Linie im Ergebnisbild breiter ist.

Anwendung auf Digitale Höhenmodelle und kombinierter Einsatz Die Konzepte der Erosion und Dilatation sind nicht auf Binärbilder beschränkt, sondern lassen sich auch auf Grauwertbilder erweitern und damit können sie auch bei einem Digitalen Höhenmodell (DHM) angewendet werden, da es ein eindimensionales Band von Pixeln ist.

- Bei der Grauwert-Erosion wird der Helligkeitswert eines Pixels durch den minimalen Intensitätswert ersetzt, der sich in der durch das strukturierende Element definierten Nachbarschaft findet.
- Bei der Grauwert-Dilatation hingegen wird der Helligkeitswert eines Pixels durch den maximalen Intensitätswert aus dieser Nachbarschaft ersetzt.

Für sich allein genommen ist der Nutzen von Erosion und Dilatation oft begrenzt. In Kombination können sie zur Ableitung von höherwertigen Funktionen verwendet werden. Abbildung 3.3 zeigt, wie Erosion und Dilatation angewendet werden können, um potenzielle Gipfel zu erkennen.

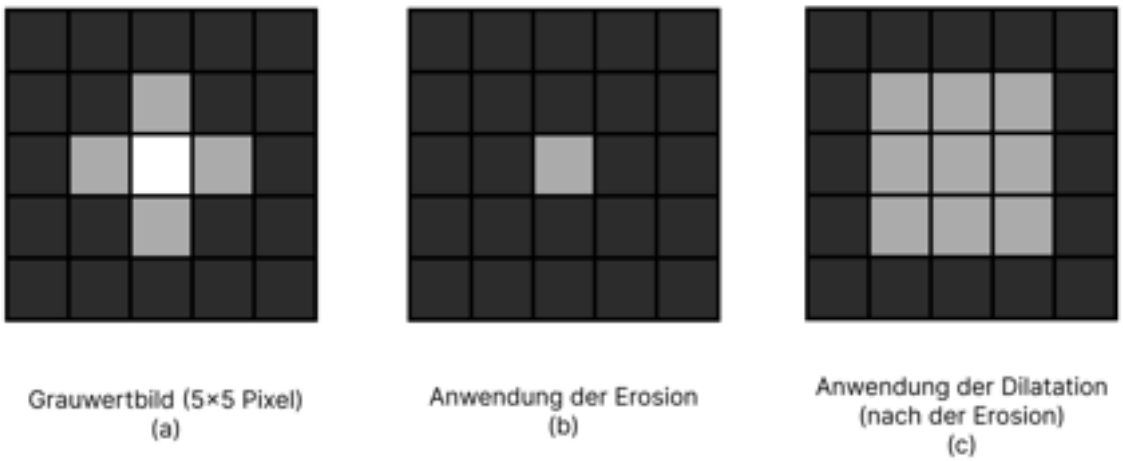

Abbildung 3.3: Beispielsergebnis der Anwendung der kombinierten morphologischen Operationen

Bei (a) ist ein Grauwertbild dargestellt, welches eine kleine Erhebung darstellt. Um zunächst nur die wesentlichen Pixel dieser Erhebung zu extrahieren und Rauschen zu entfernen, wird die Erosion angewendet. Das Ergebnis dieser Erosion ist bei (b) zu sehen, wo die Erhebung im Vergleich zum Originalbild verkleinert erscheint. Anschließend wird auf das durch Erosion bearbeitete Bild die Dilatation angewendet. Das finale Ergebnis ist bei (c) dargestellt. Durch diese Kombination von Erosion und Dilatation, der sogenannten Öffnung, können schwerer erkennbare oder verrauschte Erhebungen oder Objekte in einem Bild deutlicher und klarer hervorgehoben werden. [27]

3.2.2 Lokaler Maximum-Filter

Der Maximum-Filter stellt ein grundlegendes Verfahren innerhalb der Bildverarbeitung dar. Die Funktionsweise dieses Filters beruht auf der Ermittlung des höchsten Pixelintensitätswerts innerhalb einer definierten Nachbarschaft. Dabei wird der Intensitätswert des zentralen Pixels durch den Maximalwert aller Pixel innerhalb der Nachbarschaft (z.B. 3x3), ersetzt. Dieser Vorgang wird iterativ für jedes Pixel im Bild durchgeführt. [27] Abbildung 3.4 zeigt, wie der Maximum-Filter eingesetzt werden kann, um die Position der lokalen Maxima eines Bildes zu bestimmen.

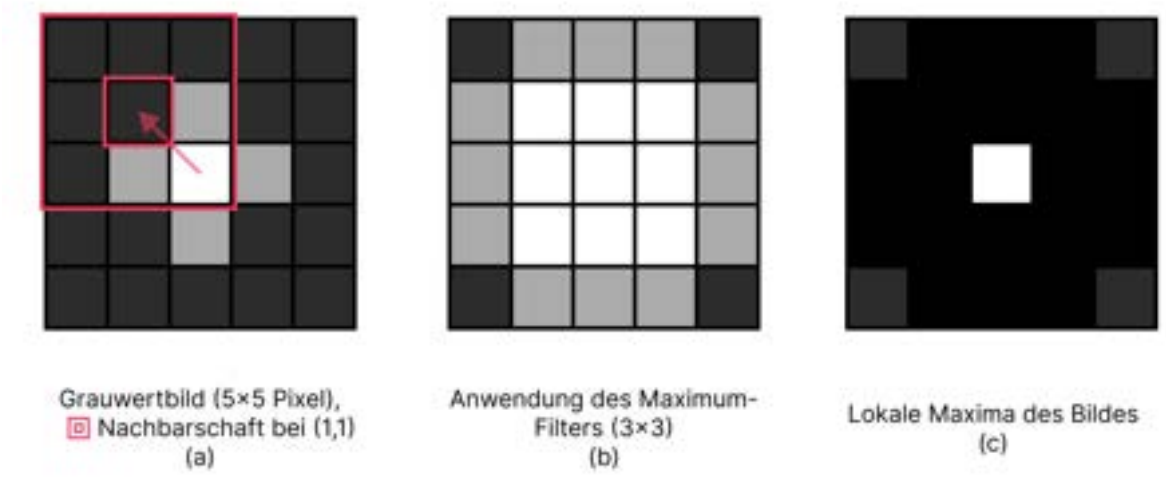

Abbildung 3.4: Beispielsergebnis der Anwendung des Maximum-Filters

Bei (a) ist ein Grauwertbild dargestellt, das eine kleine Erhebung zeigt. Zur Veranschaulichung ist zusätzlich die 3x3-Nachbarschaft um den Pixel an der Koordinate (1,1) eingezeichnet. Im Beispiel wird die Intensität dieses Pixels durch die des links-unteren Pixels der Nachbarschaft (Weiß) überschrieben. Das Ergebnis der vollständigen Anwendung des Maximum-Filters auf das gesamte Bild ist bei (b) zu sehen. Anschließend wird überprüft, an welchen Stellen die Pixelintensität im gefilterten Bild mit der Intensität im Originalbild übereinstimmt. Damit lassen sich die lokalen Maxima des Bildes identifizieren. Diese Methode kann analog auf DHMs angewendet werden, um die potenziellen Gipfel zu bestimmen.

3.2.3 Vergleich von Morphologische Operation und Maximum-Filter

Die morphologische Öffnung ist nützlich, wenn die Daten verrauscht sind oder viele kleine, irrelevante Erhebungen enthalten, die ignoriert werden sollen. Sie glättet das Bild und hilft, die signifikanten Gipfel von der Masse kleinerer Unregelmäßigkeiten zu trennen. Das Ergebnis ist eine Art bereinigte Topografie, auf der die relevanten Gipfel deutlicher hervortreten. Aber die morphologischen Operationen sind stark von der Wahl der Strukturierungselemente abhängig und die Position von den potenziellen Gipfeln sind nicht direkt ableitbar.

Der lokale Maximum-Filter ist der direktere und präzisere Ansatz, um die exakten Pixelpositionen von lokalen Maxima zu finden. Durch den direkten Vergleich des gefilterten Bildes mit dem Originalbild lassen sich die Pixel als lokale Maxima festlegen, die dann die potenziellen Gipfel darstellen. Maximum-Filter sind zusätzlich einfach zu implementieren und recheneffizient, jedoch anfällig für „falsche" Gipfel, die durch Rauschen oder lokale Unebenheiten entstehen, und berücksichtigen keine überregionale Bedeutung.

Für die vorläufige Gipfelbestimmung der Maximum-Filter gewählt. Diese Entscheidung basiert auf seiner Effizienz und der Fähigkeit, die genauen Koordinaten der lokalen Maxima zu liefern. Obwohl dieser Ansatz anfänglich eine erhöhte Anzahl an Gipfelkandidaten generieren kann, beeinflusst dies die nachfolgenden Analysen der Prominenz- und Dominanzberechnung nicht wesentlich.

3.3 Algorithmen zur Berechnung der Prominenz

Nach der vorläufigen Identifizierung potenzieller Gipfel ist der nächste Schritt die Bestimmung ihrer Prominenz. Für diese Prominenzberechnung kommen primär zwei Algorithmen infrage: der Bresenham-Algorithmus und der Dijkstra-Algorithmus. Im Folgenden werden diese beiden Algorithmen beschrieben und anschließend hinsichtlich ihrer Eignung für die Prominenzbestimmung verglichen.

3.3.1 Bresenham-Algorithmus

Der Bresenham-Algorithmus wurde um 1965 von J. E. Bresenham bei IBM entwickelt. Die primäre Motivation hinter der Entwicklung des Algorithmus war es, eine Methode zur Rasterung einer Linie zu schaffen, die ausschließlich auf Integer-Addition und -Subtraktion basiert. Die exakte Berechnung der y-Position eines Punktes auf einer Linie für einen gegebenen x-Wert erfordert typischerweise die Verwendung der Steigung $m = \frac{\Delta y}{\Delta x}$. Dies würde eine Gleitkommaberechnung unumgänglich machen. [28]

Der Bresenham-Algorithmus umgeht diese Notwendigkeit, indem er direkt mit den ganzzahligen Differenzen Δy und Δx arbeitet. Es schreitet entlang einer Achse (z.B. x-Achse) iterativ voran und trifft bei jedem Schritt die Entscheidung, ob auch auf der anderen Achse (z.B. y-Achse) ein Schritt ausgeführt werden muss. Die Entscheidung, ob ein Schritt auf der sekundären Achse vollzogen wird, basiert auf einer „Fehlervariable". Diese Variable ist ein Wert, der den akkumulierten Fehler zwischen der idealen Linienposition und der tatsächlichen Pixelposition repräsentiert. [28] Der Algorithmus lässt sich wie folgt beschreiben:

1. Zuerst werden die absoluten Differenzen $\Delta x = |x1 - x0|$ und $\Delta y = |y1 - y0|$ berechnet. Die Achse mit der größeren Differenz wird zur primären Achse. Entlang dieser Achse wird in jeder einzelnen Iteration eine Bewegung ausgeführt.

2. Eine Fehlervariable p wird einmalig vor dem ersten Schritt berechnet. Bei jedem Schritt entlang der primären Achse wird diese Variable dann itera-

tiv aktualisiert, nicht komplett neu bestimmt. Diese Aktualisierung ist eine einfache Addition.

3. Ist die Fehlervariable größer gleich Null ($p \geq 0$), wird ein Schritt auf der sekundären Achse (z.B. y) ausgeführt.

Durch dieses Vorgehen wird die Linie schrittweise approximiert, ohne dass eine Gleitkomma-Operation oder Multiplikation notwendig ist. [28] Abbildung 3.5 zeigt ein Beispiel des Bresenham-Algorithmus, das den Pfad von (0, 0) zu (5, 4) berechnet:

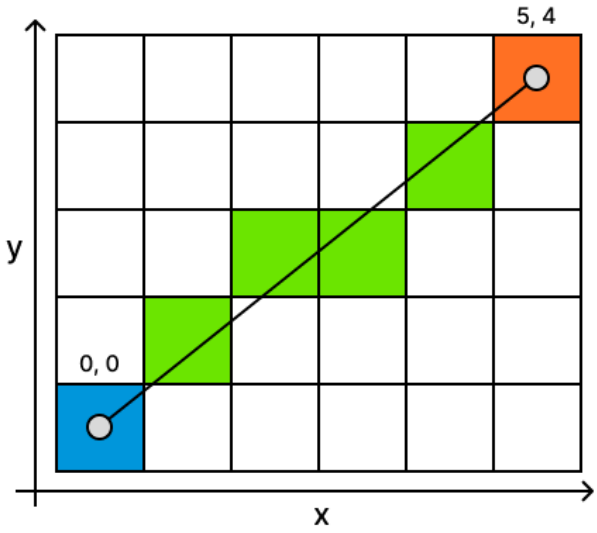

Abbildung 3.5: Beispiel für eine Linie nach dem Bresenham-Algorithmus

Der Pixel in Blau markiert den Startpunkt der Linie, der Pixel in Orange den Endpunkt. Da in die Differenz zur x-Richtung größer ist, ist die x-Achse die primäre Achse. Für jede Iteration wird x nun inkrementiert und überprüft, ob y ebenfalls inkrementiert werden muss. Die Sequenz der berechneten Pixel für die Linie von (0,0) bis (5,4) lautet: (0,0)→(1,1)→(2,2)→(3,2)→(4,3)→(5,4).

Mit dem Bresenham-Algorithmus lässt sich die Prominenz eines Gipfels näherungsweise bestimmen. Dazu wird zunächst mit dem Algorithmus ein linearer Pfad vom Gipfel zu einem nächstgelegenen, höheren Gipfel berechnet. Entlang dieses Pfads wird dann der tiefste Punkt ermittelt. Die Differenz zwischen der Höhe des Gipfels und diesem tiefsten Punkt ist dann die Schartenhöhe.

3.3.2 Dijkstra-Algorithmus

Der Dijkstra-Algorithmus wurde in den 1950er Jahren entwickelt und ermöglicht es, den kürzesten Pfad zwischen einem Knoten zu allen anderen Knoten zu finden. [29] Der Algorithmus arbeitet auf einen gewichteten Graphen. Ein solcher Graph ist definiert durch:

- Eine endliche Menge von Knoten.
- Eine Menge von Kanten, wobei jede Kante (u, v) eine Verbindung von einem Knoten u zu einem anderen Knoten v darstellt.
- Eine Menge von Gewichten, die zu einer Kante zugeordnet ist und die „Kosten" der Kante darstellt und nicht-negativ ist.

Der Dijkstra-Algorithmus beginnt mit der Initialisierung der Distanzschätzungen. Dem Startknoten s wird ein initialer Wert von $d(s) = 0$ zugewiesen. Für alle anderen Knoten v wird der Wert von $d(v) = \infty$ gesetzt. Diese Werte ($d(v)$), dienen als laufende Schätzungen der kürzesten Distanz vom Startknoten s zu jedem Knoten v. [30] Der Kern des Algorithmus besteht aus einer Schleife, die fortgesetzt wird, bis alle Knoten „besucht" wurden:

1. **Knotenauswahl:** In jeder Iteration wird von allen nichtbesuchten Knoten derjenige Knoten u ausgewählt, dessen Wert $d(u)$ am kleinsten ist. Der ausgewählte Knoten u wird dann als „besucht" markiert. Am Anfang wird der Startknoten s ausgewählt, da dessen Wert $d(s)$ am kleinsten ist. [30]
2. **Aktualisierung der Nachbarknoten:** Für jede ausgehende Kante (u, v) vom Knoten u werden die Werte der nichtbesuchten Nachbarknoten v nach dieser Regel aktualisiert:
$$d(v) = min\{d(v), d(u) + w(u, v)\}$$
Das bedeutet, dass der Wert von $d(v)$ aktualisiert wird, wenn der $d(u)$ addiert mit $w(u, v)$ (das Gewicht der Kante zwischen u und v), kleiner ist als $d(v)$. [30]
3. **Aktivierung neuer Knoten:** Falls ein Nachbarknoten v zuvor „besucht" war, wird er nach der Aktualisierung seines Wertes als Ausgangknoten für die nächste Iteration verwendet. [30]

Nachdem alle Knoten besucht wurden, enthalten die Werte $d(v)$ für jeden Knoten v den gesuchten Wert (kürzeste Distanz) vom Startknoten s zu v. Abbildung 3.6 zeigt ein Beispiel eines kürzesten Pfades nach dem Dijkstra-Algorithmus:

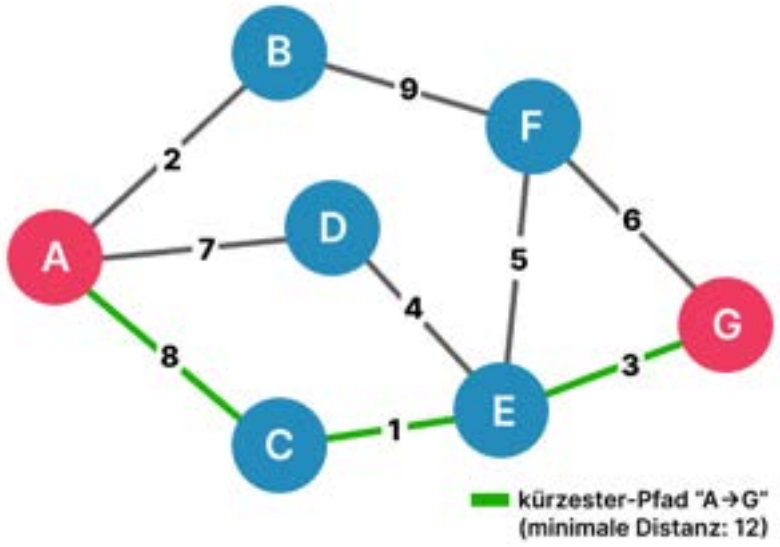

Abbildung 3.6: Beispiel eines Pfades nach dem Dijkstra-Algorithmus

Hierbei ist A der Startknoten und G der Endknoten. Im Beispiel ist A→C→E→G der Pfad, wo die Distanz minimal ist. Alle anderen möglichen Pfade überschreiten die Distanz von 12.

Für die Prominenzberechnung kann der normale Dijkstra-Algorithmus nicht verwendet werden, da nicht der kürzeste Pfad zwischen Gipfeln gesucht ist. Gesucht ist der Pfad, bei dem das kleinste Gewicht (Tiefpunkt) einer einzelnen Kante so groß (hoch) wie möglich ist. Deshalb wird eine modifizierte Version des Dijkstra-Algorithmus verwendet.

Der Max-Min-Dijkstra-Algorithmus, ist eine Modifikation des klassischen Dijkstra-Algorithmus. Sein Ziel ist es, für einen gegebenen Startknoten den Pfad zu finden, dessen Minimum über alle möglichen Pfade hinweg maximal ist. [31]

Der Max-Min Dijkstra-Algorithmus unterscheidet sich nur geringfügig zu dem normalen Dijkstra-Algorithmus. Der Hauptunterschied ist die Aktualisierungsregel innerhalb der Schleife:

$$d(v) = min\{d(v), w(u, v)\}$$

Diese Regel implementiert das Max-Min-Kriterium. Das bedeutet, dass der Wert von d(v) aktualisiert wird, wenn das Gewicht des Pfades zwischen u und v (w(u, v)), kleiner ist als d(v) [31]. Abbildung 3.7 zeigt ein Beispiel eines Max-Min Pfades:

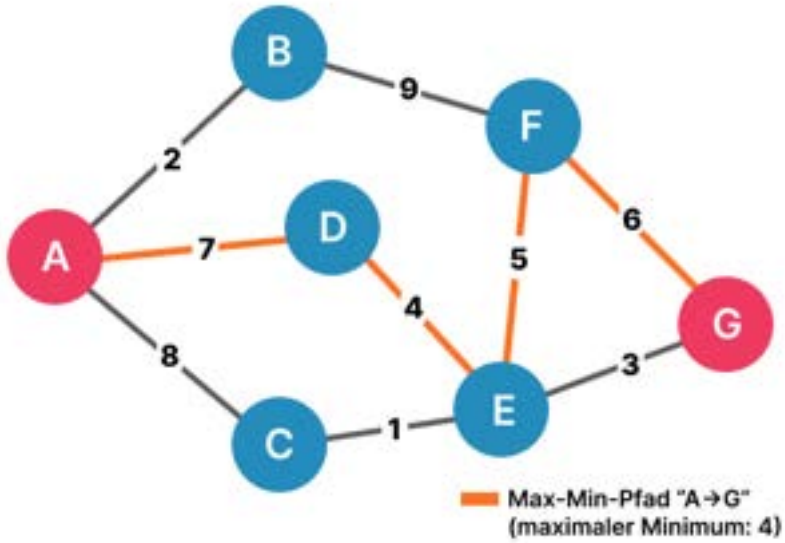

Abbildung 3.7: Beispiel eines Pfades nach dem Max-Min-Dijkstra-Algorithmus

Hierbei ist A der Startknoten und G der Endknoten. Im Beispiel ist A→D→E→F→G der Pfad, wo das Minimum des Pfades maximal ist. Alle anderen möglichen Pfade von A nach G gehen durch eine Kante das kleiner ist als 4. Wenn man A und G als Gipfel betrachtet und die Kantengewichte als Höhen, dann müsste man mindestens auf eine Höhe von 4 Einheiten herabsteigen, um von Gipfel A nach G zu kommen.

3.3.3 Vergleich zwischen Bresenham- und Dijkstra-Algorithmus

Der Bresenham-Algorithmus ist primär für die Linienrasterung konzipiert und bietet eine sehr effiziente, ganzzahlige Methode zur Berechnung eines Pfades zwischen zwei Punkten. Für die Prominenzbestimmung wird ein linearer Pfad vom Gipfel zum nächstgelegenen höheren Punkt erzeugt, entlang dessen der tiefste Punkt (Sattelpunkt) gesucht wird. Der Vorteil liegt in der rechnerischen Einfachheit und Geschwindigkeit, besonders auf Rasterdaten. Allerdings approximiert er lediglich eine gerade Linie und berücksichtigt nicht das tatsächliche Gelände

zwischen den Punkten. Der ermittelte Sattelpunkt ist daher nur eine grobe Annäherung, da der reale Pfad durch das Gelände selten linear ist und möglicherweise tiefere Punkte außerhalb der Bresenham-Linie existieren.

Im Gegensatz dazu ist der Max-Min-Dijkstra-Algorithmus speziell darauf ausgelegt, den Pfad mit dem höchsten Sattelpunkt zu finden. Der Max-Min-Dijkstra navigiert durch die tatsächlichen Geländepunkte und berücksichtigt alle möglichen Pfade, um den optimalen Punkt zu finden. Dies führt zu einer genauen Bestimmung der Schartenhöhe. Der Nachteil des Max-Min-Dijkstra-Algorithmus liegt in seiner hohen Rechenintensität, das er im Vergleich zum Bresenham-Algorithmus deutlich langsamer ist. Die mögliche Abweichung bei der Prominenzberechnung bei Nutzung des Bresenham-Algorithmus wird in Abbildung 3.8 veranschaulicht:

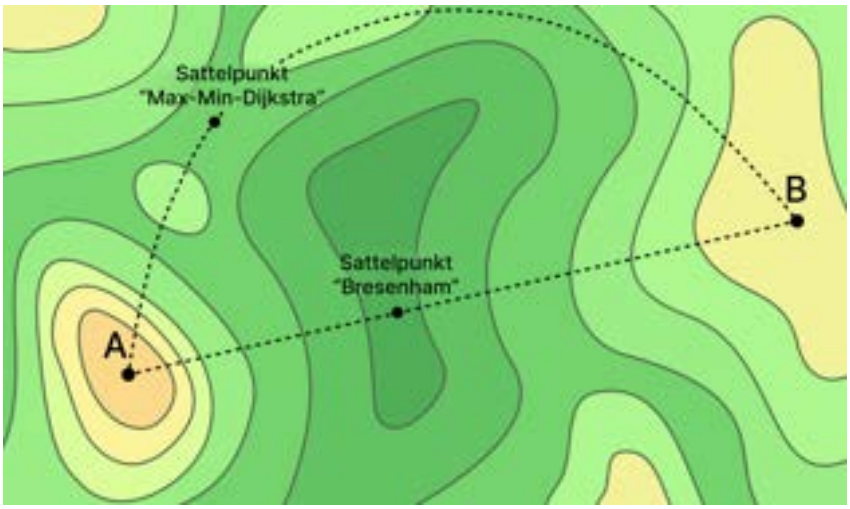

Abbildung 3.8: Abweichung der Prominenzberechnung mit dem Bresenham-Algorithmus

Das Beispiel zeigt eine topografische Darstellung mit zwei Gipfeln, Gipfel A und Gipfel B. Auch sind zwei unterschiedliche Pfade, die diese Gipfel verbinden dargestellt: ein linearer Pfad, berechnet mit dem Bresenham-Algorithmus, und ein optimaler Pfad, berechnet mit dem Max-Min-Dijkstra-Algorithmus. Für jeden Pfad ist der zugehörige tiefste Punkt markiert. Es ist deutlich zu erkennen, dass der Tiefpunkt des Bresenham-Pfades tiefer liegt als der Tiefpunkt des Pfades, der mit dem Max-Min-Dijkstra-Algorithmus ermittelt wurde. Dies hat zur Folge, dass

die mit dem Bresenham-Algorithmus berechnete Prominenz des Gipfels überschätzt wird, da sie nicht den höchsten möglichen Tiefpunkt widerspiegelt. Die so ermittelte Prominenz ist demnach stets höher als die tatsächliche Prominenz, die den optimalen Pfad berücksichtigt.

Diese Überschätzung der Prominenz durch den Bresenham-Algorithmus lässt sich strategisch nutzen: Jeder Gipfel, der durch die präzisere Dijkstra-Berechnung als prominent eingestuft wird (d.h., seine Prominenz liegt über einen Schwellenwert), wird auch mit dem Bresenham-Algorithmus als prominent erkannt. Das ermöglicht einen zweistufigen Ansatz:

- Zuerst wird der effizientere Bresenham-Algorithmus für eine Vorfilterung der potenziellen Gipfel eingesetzt. Dies reduziert die anfänglich große Menge an potenziellen Gipfeln erheblich.
- Anschließend wird der rechenintensivere Dijkstra-Algorithmus nur noch auf diese stark reduzierte Menge von Gipfelkandidaten angewendet, um deren endgültige, genaue Prominenzwerte zu bestimmen.

Dieser Ansatz führt zu einer signifikanten Beschleunigung der gesamten Analyse, ohne die Genauigkeit der finalen Prominenzwerte zu beeinträchtigen. Daher wird dieser zweistufige Ansatz im Rahmen dieser Arbeit verfolgt.

3.4 Algorithmen zur Berechnung der Dominanz

Da es sich bei der Dominanz um Abstände handelt, gibt es nur wenige Methoden zu ihrer Berechnung. In der Bildverarbeitung wird für den Abstand zweier Punkte der euklidische Abstand verwendet [27]:

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Ein geeigneter Ansatz zur Bestimmung der Dominanz besteht in der Verwendung der euklidischen Distanztransformation. Diese Transformation berechnet für jedes Pixel innerhalb einer definierten Maske den kürzesten euklidischen Abstand zum nächstgelegenen Zielpixel. [32]

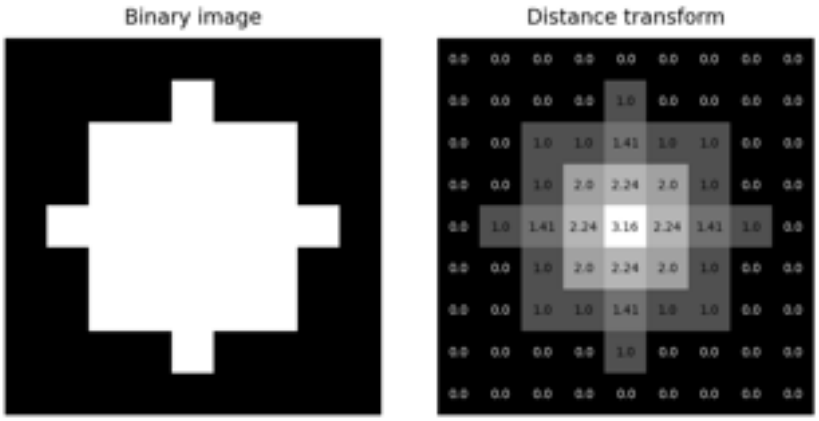

Abbildung 3.9: Binäre Maske und die korrespondierende euklidische Distanztransformation [32]

Stelle man sich beispielsweise den mittleren Punkt des Bildes als Gipfel vor, wobei die schwarzen Pixel die höher gelegenen Bereiche darstellen. Die euklidischen Distanztransformation würde dann für diesen Gipfel eine Dominanz von etwa 3.16 Pixeleinheiten liefern. Analog lässt sich die Dominanz der Erhebungen in DHMs auf diese Weise bestimmen.

3.5 Fazit

Das Kapitel 3 vergleicht mehrere Implementierungsansätze zur Gipfelfindung. Unter den verfügbaren Verfahren wird die Methode der Filterung nach Prominenz und Dominanz ausgewählt. Ausschlaggebend für diese Entscheidung waren die hohe Steuerbarkeit des Verfahrens sowie der geringe mittlere Distanzfehler. Für die erste Identifikation potenzieller Gipfelkandidaten kommen morphologische Operationen und der lokale Maximum-Filter infrage. Der lokale Maximum-Filter wird bevorzugt, da er effizient arbeitet und die Position lokaler Maxima direkt ermittelt. Die Berechnung der Prominenz erfolgt in zwei Schritten. Zunächst reduziert der Bresenham-Algorithmus die Anzahl der relevanten Kandidaten durch eine schnelle Vorfilterung. Danach bestimmt der Max-Min-Dijkstra-Algorithmus die tatsächliche Prominenz der verbleibenden Punkte. Diese Kombination vereint die Vorteile beider Verfahren in Bezug auf Effizienz und Genauigkeit. Die Dominanz wird mithilfe der euklidischen Distanztransformation berechnet. Dieses Verfahren ermöglicht für jedes Pixel eine effiziente Bestimmung der Distanz zum nächsthöheren Gipfel.

4 Implementierung der automatischen Gipfelerkennung

Dieses Kapitel behandelt die praktische Entwicklung der Softwarelösung für die automatische Gipfelerkennung. Zunächst werden die funktionalen und nichtfunktionalen Anforderungen für das Projekt aufgestellt und die zu verwendenden Technologien ausgewählt. Es folgt die Architektur der Gesamtlösung, ehe die Implementierung der vier Hauptkomponenten im Detail dargestellt wird. Zuletzt erfolgt die Validierung der erstellten Softwarelösung und ein kurzes Zwischenfazit.

4.1 Anforderungen

Für die Softwarelösung werden einige funktionale und nichtfunktionale Anforderungen aufgestellt. Sie sind in den folgenden Unterkapiteln 4.1.1 und 4.1.2 gelistet.

4.1.1 Funktionale Anforderungen

Die Hauptfunktionalität der Softwarelösung ist die Identifizierung und Darstellung von Gipfeln aus DHMs im GeoTIF-Format. Darüber hinaus werden folgende funktionale Anforderungen definiert:

1. **Einlesen von GeoTIFF-Dateien:** Die Software soll in der Lage sein, GeoTIFF-Dateien unterschiedlicher Größe und Auflösung fehlerfrei einzulesen und die darin enthaltenen Höhendaten zu verarbeiten.

2. **Gipfelerkennung:** Basierend auf den eingelesenen Höhendaten soll die Software Gipfel automatisch identifizieren können. Dabei sollen verschiedene Kriterien zur Gipfeldefinition berücksichtigt werden, insbesondere die Prominenz und Dominanz.
3. **Benutzerfreundliche Oberfläche (GUI):** Die Software soll über eine intuitive grafische Benutzeroberfläche verfügen, die es den Nutzern ermöglicht, GeoTIFF-Dateien einfach hochzuladen und zu öffnen. Außerdem sollen die Parameter für die Gipfelerkennung (z.B. Prominenz) anpassbar sein.
4. **Grafische Darstellung der Gipfel:** Die gefundenen Gipfel sollen in einer grafischen Ausgabemaske visualisiert werden.
5. **Koordinatenausgabe:** Für jeden identifizierten Gipfel sollen die geografischen Koordinaten, beispielsweise Längengrad und Breitengrad, angezeigt werden.

4.1.2 Nichtfunktionale Anforderungen

Neben den funktionalen Anforderungen werden auch einige nichtfunktionale Anforderungen für die zu erstellende Softwarelösung aufgestellt:

1. **Leistungsfähigkeit:** Die Software soll eine gute Leistungsfähigkeit aufweisen. Dies bedeutet, dass das Einlesen der GeoTIFF-Dateien, die Durchführung der Gipfelerkennung und die Visualisierung der Ergebnisse in einer angemessenen Zeit erfolgen soll.
2. **Benutzererfahrung (UX):** Die Software soll eine intuitive und benutzerfreundliche Bedienung ermöglichen.
3. **Genauigkeit:** Die Ergebnisse der Gipfelerkennung, insbesondere die Positionen und die abgeleiteten Eigenschaften (z.B. Prominenz, Dominanz), müssen so genau wie möglich sein, basierend auf der Auflösung und Qualität der Eingabedaten.
4. **Skalierbarkeit für große Datenmengen:** Die Software muss in der Lage sein, auch große GeoTIFF-Dateien effizient zu verarbeiten, ohne signifikante Leistungseinbußen oder Speicherprobleme zu verursachen.

4.2 Auswahl der Technologien

Um die in Kapitel 4.1 beschriebenen Anforderungen zu implementieren, bedarf es geeigneten Technologien. In den folgenden Abschnitten wird die Auswahl geeigneter Programmiersprachen und Bibliotheken für die Geodatenverarbeitung, die Berechnung von Gipfeln sowie für die Entwicklung der grafischen Benutzeroberfläche diskutiert.

4.2.1 Programmiersprache

Für topografische Analysen und Gipfelidentifikationen wird eine Programmiersprache benötigt, die auf Geodatenformate zugreifen kann und eine Geospatial Data Abstraction Library (GDAL)-Schnittstelle ermöglicht. GDAL ist eine Bibliothek zur Verarbeitung verschiedenster Geodatenformate. Sie bietet aktuell nur Schnittstellen für C/C++ und Python, weshalb sich die Auswahl der Programmiersprache auf diese beiden Optionen beschränkt. [33] Obwohl C/C++ eine hohe Performance und Speichereffizienz aufweist [34], fällt die Wahl für die Entwicklung der Softwarelösung auf Python. Das hat folgende Gründe: Ein wesentlicher Faktor ist die Verfügbarkeit leistungsstarker wissenschaftlicher Bibliotheken wie NumPy oder SciPy. Ihre Funktionalitäten werden in Kapitel 4.2.3 näher erläutert. Darüber hinaus existieren Bibliotheken wie Numba, die durch Optimierungen eine Performance ermöglichen, die der von C/C++ nahekommt. [35] Der potenzielle Nachteil der langsameren Ausführungsgeschwindigkeit von Python wird somit effektiv kompensiert.

4.2.2 Bibliotheken für die Geodatenverarbeitung

Folgende Python-Bibliotheken werden für die Geodatenverarbeitung verwendet:

GDAL GDAL ist eine fundamentale Bibliothek, die verschiedene Raster- und Vektordatenformaten unterstützt. Sie ermöglicht das Lesen, Schreiben, Bearbeiten und die Durchführung grundlegender geodätischer Analysen [33].

Rasterio Aufbauend auf GDAL bietet Rasterio eine benutzerfreundliche Programmierschnittstelle für die Verarbeitung von Rasterdaten. Durch die Integration mit der numerischen Bibliothek NumPy ermöglicht Rasterio eine Array-basierte Verarbeitung von Rasterdaten und einen einfachen Zugriff auf Metadaten von GeoTIFF-Dateien und anderen Rasterformaten [36].

pyproj Diese Bibliothek ermöglicht die Konvertierung von Koordinaten zwischen verschiedenen geografischen Koordinatensystemen und unterstützt wichtige geodätische Berechnungen [37]. Da die Verarbeitung von Geodaten aus unterschiedlichen Quellen oft unterschiedliche Koordinatensysteme involviert, ist pyproj für die Genauigkeit und Konsistenz der Analysen notwendig.

4.2.3 Bibliotheken für die Gipfelberechnung

Folgende Python-Bibliotheken werden für die Gipfelberechnung verwendet:

NumPy Numerical Python (NumPy) ist eine Kernbibliothek für numerische Berechnungen in Python. Seine Array-Objekte und mathematischen Funktionen [38] bilden die Grundlage für die effiziente Verarbeitung großer Rasterdatensätze in Bibliotheken wie Rasterio und vereinfachen die Implementierung komplexer topografischer Algorithmen.

SciPy Scientific Python (SciPy) erweitert die Funktionalität von NumPy um eine Vielzahl wissenschaftlicher und technischer Algorithmen, darunter Funktionen für Signalverarbeitung, räumliche Datenstrukturen und Optimierungsverfahren. [39]

scikit-image Diese vielseitige Bildverarbeitungsbibliothek bietet Algorithmen für verschiedene Aufgaben wie Filterung, Segmentierung und geometrische Transformationen. [40]

heapq Als Teil der Python-Standardbibliothek implementiert `heapq` den Heap-Warteschlangen-Algorithmus zur Realisierung von Prioritätswarteschlangen. Es wird für die effiziente Implementierung des modifizierten Dijkstra-Algorithmus zur Berechnung der Schartenhöhe verwendet. [41]

Numba Numba ist ein Just-In-Time (JIT)-Compiler für Python. Es kann Python-Funktionen, die viele numerische Berechnungen durchführen, in optimierten Maschinencode umwandeln, oft mit Geschwindigkeiten, die denen von C oder Fortran nahekommen [35]. Dies ist besonders nützlich für rechenintensive Schleifen und Algorithmen, die direkt auf NumPy-Arrays operieren und die Performance der Gipfelberechnung erheblich verbessern können, ohne dass man auf eine andere Sprache zurückgreifen muss.

4.2.4 Bibliotheken für die grafische Benutzeroberfläche

Für die Entwicklung der Grafical User Interface (GUI) stehen in Python mehrere etablierte Technologien zur Auswahl. Zu den Bekanntesten zählen PyQt und Tkinter, die jeweils Vor- und Nachteile bieten und in den folgenden Abschnitten vorgestellt werden.

PyQt PyQt ist ein mächtiges Framework, das eine sehr umfangreiche Sammlung an UI-Elementen und fortgeschrittenen Anpassungsmöglichkeiten bereitstellt [42]. Es erlaubt die Entwicklung komplexer und professionell aussehender Anwendungen. Allerdings kann der Funktionsumfang für einfachere Anwendungen einen unnötigen Overhead bedeuten. Zudem ist für die kommerzielle Nutzung von PyQt eine entsprechende Lizenz erforderlich [42], was ein wichtiger Kostenfaktor sein kann.

Tkinter Tkinter hingegen ist eine in Python standardmäßig enthaltene Bibliothek und zeichnet sich durch ihre einfache Erlernbarkeit und ihren minimalistischen Ansatz aus [43]. Dies ermöglicht einen schnellen Einstieg in die GUI-Entwicklung und ist für unkompliziertere Anwendungen gut geeignet. Die Stan-

dardausführung von Tkinter bietet jedoch begrenzte Möglichkeiten zur optischen Anpassung und kann bei modernen Designansprüchen an ihre Grenzen stoßen.

CustomTkinter Eine vielversprechende Alternative stellt hier CustomTkinter dar. Diese Bibliothek baut auf Tkinter auf und erweitert dessen Funktionalität signifikant um moderne Designelemente und zusätzliche UI-Komponenten [44]. Dadurch bietet CustomTkinter einen attraktiven Kompromiss zwischen der Einfachheit und der breiten Verfügbarkeit von Tkinter und der Möglichkeit, ansprechendere und flexiblere Benutzeroberflächen zu gestalten.

Matplotlib Matplotlib ist eine weit verbreitete und flexible Bibliothek zur Erstellung von statischen, interaktiven und animierten Visualisierungen in Python [45]. Durch die Integration von Matplotlib in die Tkinter-GUI ist es möglich, Karten als Plots direkt in der Anwendung anzuzeigen und so eine visuelle Rückmeldung der Analyseergebnisse zu ermöglichen.

Aufgrund der Notwendigkeit einer benutzerfreundlichen Oberfläche, ohne die Komplexität und potenziellen Lizenzkosten von PyQt in Kauf nehmen zu müssen, fällt die Wahl für die Entwicklung der GUI auf Tkinter in Kombination mit CustomTkinter für erweiterte Designmöglichkeiten. Für die Visualisierung der Kartendaten und der Analyseergebnisse wird Matplotlib in die Tkinter-Anwendung integriert.

4.3 Architektur

Für das Projekt wird eine modulare Architektur mit getrennten Dateien gewählt. Hintergrund ist die Trennung von Zuständigkeiten, sodass jede Datei ausschließlich ihre spezifische Aufgabe erfüllt und insbesondere Logik und Visualisierung strikt voneinander getrennt sind. Der Aufbau ermöglicht eine klare Strukturierung und erleichtert die Entwicklung und das Testing. Zudem ermöglicht es mögliche Erweiterungen in der Zukunft. Abbildung 4.1 stellt die Architektur grafisch dar:

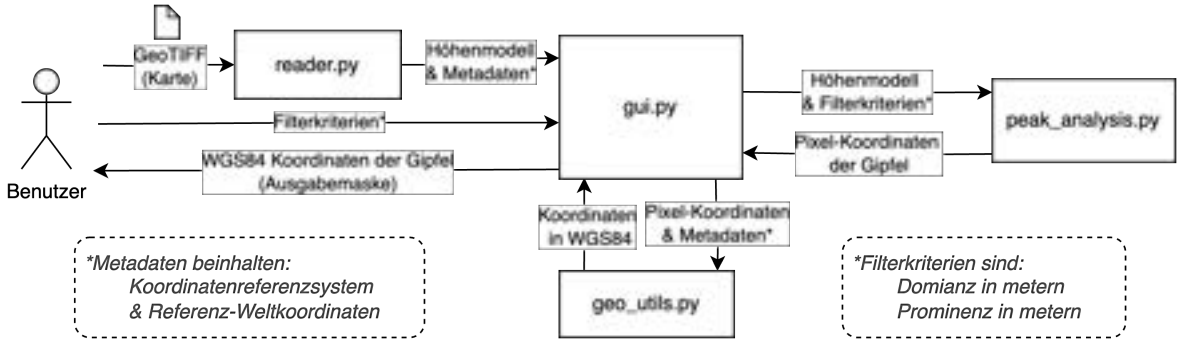

Abbildung 4.1: Aufbau des Programms (nur wesentliche Funktionen)

Es handelt sich vorrangig um vier Hauptkomponenten, deren Funktionalität in den nachfolgenden Abschnitten in Kürze vorgestellt wird. Die Kapitel 4.4 bis 4.7 beschäftigen sich anschließend ausführlich mit der Implementierung der Komponenten.

reader.py Diese Komponente ist primär für das Einlesen von GeoTIFF-Dateien zuständig. Sie extrahiert sowohl die Rasterdaten als auch die relevanten Metadaten aus dem eingegebenDHM im GeoTIF-Format.

gui.py Bei dieser Komponente handelt es sich um das Herzstück der Anwendung. Sie implementiert die grafische Benutzeroberfläche und fungiert als Vermittler zwischen anderen Komponenten. Sie ermöglicht dem Nutzer das Hochladen der DHMs in Form von GeoTIFF-Dateien. Zudem können hier die Parameter für die Gipfelerkennung, wie beispielsweise der Prominenz und Dominanz eingestellt

werden. Die Ergebnisse der Gipfelberechnung werden ebenfalls über die gui.py dargestellt.

geo_utils.py Dieses Modul beinhaltet Funktionen für georäumliche Operationen. Dazu gehört die Transformation der Koordinaten der GeoTIFF-Datei in das einheitliche WGS84-Koordinatensystem. Weiterhin ist sie für die Verarbeitung der Kartenauflösung zuständig. Es ermöglicht die Umrechnung einer in Metern angegebenen Dominanzweite in die entsprechende Pixelanzahl, basierend auf der Auflösung der GeoTIFF-Datei. Diese Funktionalität ist entscheidend für die korrekte Anwendung des Dominanzkriteriums bei der Gipfelerkennung.

peak_analysis.py Die Kernlogik zur Identifizierung der Gipfel aus den Rasterdaten des eingegebenen DHMs wird in `peak_analysis.py` implementiert. Die enthaltenen Algorithmen werden auf die Höhendaten angewendet, um Gipfel basierend auf den definierten Kriterien (z.B. Prominenz und Dominanz) zu ermitteln.

4.4 Implementierung der Datenextraktion (reader.py)

Wie in Kapitel 4.3 beschrieben, wird das **`reader.py`**-Modul für das Einlesen der GeoTIFF-Datei verwendet. Es beinhaltet die Methode **`read_dem`**, welche die Rasterdaten des DHMs sowie die zugehörigen Metadaten aus der GeoTIFF-Datei extrahiert.

4.4.1 Auslesen der Rasterdaten

Wie in Kapitel 2.2.6 beschrieben, unterstützen GeoTIFF-Dateien die Multiband-Unterstützung. Die Höhendaten sind dabei meist auf dem ersten Band gespeichert. Die Bibliothek **`Rasterio`** bietet eine einfache Möglichkeit, auf diese Daten zuzugreifen. Quellcode 4.1 zeigt die Extraktion der Höhendaten in ein NumPy-Array:

```
import rasterio
with rasterio.open(file_path) as src:
    dem_data = src.read(1) # Liest das erste Band -> DEM
```

Quellcode 4.1: Auslesen der Höhendaten aus der GeoTIFF

Die Funktion **`open`** ermöglicht die Extraktion der Daten. Mit **`read`** können anschließend die Bänder gelesen werden, wobei das erste Band gelesen wird, um die Höhendaten zu extrahieren. [36] Die resultierende Variable **`dem_data`** enthält nun ein NumPy-Array, das die Höhendaten des Geländes repräsentiert und die Grundlage für die nachfolgenden Analysen bildet. Dieses Array ist zweidimensional aufgebaut und enthält die Höhenwerte in Metern, typischerweise als Integer-Werte. Jedes Element im Array entspricht der Höhe eines spezifischen geografischen Punktes. Ein Ausschnitt könnte beispielsweise folgendermaßen aussehen: *[[500, 550, ...], [600, 550, ...] , ...]*.

4.4.2 Auslesen der Metadaten

Neben den reinen Höhendaten werden Metadaten zur Georeferenzierung der GeoTIFF-Datei benötigt. Sie werden ausführlich im Kapitel 2.2.6 beschrieben. Für die vorliegende Arbeit sind insbesondere die folgenden Metadaten relevant:

- Das Koordinatenreferenzsystem (.crs)
- Die Transformationsmatrix (.geo_transform)
- Die Räumliche Auflösung (.res)

Die Metadaten werden hauptsächlich in der *geo_utils.py*-Komponente für geowissenschaftliche Berechnungen verwendet. Quellcode 4.2 zeigt das Auslesen der Metadaten mit Rasterio:

```
with rasterio.open(file_path) as src:
    crs_system = src.crs
    geo_transform = src.transform
    xres, yres = src.res
```

Quellcode 4.2: Auslesen der Metadaten aus der GeoTIFF

Wie im Quellcode dargestellt, ermöglicht das Einlesen über Rasterio den direkten Zugriff auf die gewünschten Parameter. [36]

4.5 Implementierung der Benutzeroberfläche (gui.py)

4.5.1 Überblick

Für eine möglichst benutzerfreundliche Benutzererfahrung wird die GUI anhand eines initialen Designentwurfs entwickelt. Der Designentwurf ist in Abbildung 4.2 dargestellt:

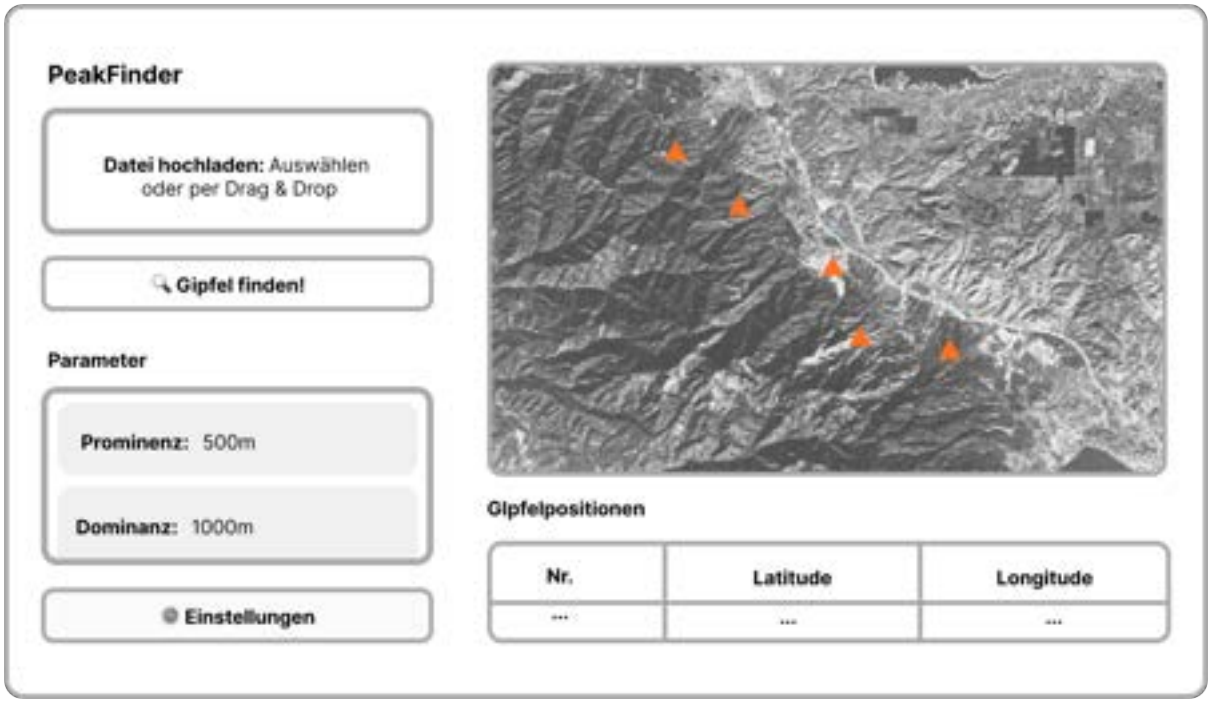

Abbildung 4.2: Designentwurf für die GUI

Auf der linken Seite befinden sich einige GUI-Elemente zur Konfiguration der Gipfelfindung. Die rechte Seite ist der Darstellung des eingegebenen DHMs und der Ergebnisse der Gipfelberechnung vorbehalten: Die erkannten Gipfel sollen final sowohl grafisch auf der Kartendarstellung als auch numerisch in der darunterliegenden Tabelle angezeigt werden. Um den Designentwurf wie oben beschrieben zu implementieren, müssen folgende technische Anforderungen umgesetzt werden:

- Der Entwurf einer geeigneten **Fensterstruktur**, um das Layout des Hauptdialogs festzulegen.

- Die Platzierung von **GUI-Elementen**, wie Eingabefelder, Bedienelemente und der Tabelle zur Darstellung der berechneten Ergebnisse.
- Die Implementierung einer Funktion zum **Hochladen des DHMs**, wodurch die GeoTIFF-Dateien über einen separaten Dialog hochgeladen werden können.
- Die **Darstellung des DHMs** als grafische Karte zur räumlichen Orientierung und der Anzeige der gefundenen Gipfel.
- Die **Darstellung der berechneten Gipfel** in der Karte sowie in der darunterliegenden Tabelle.

Die nachfolgenden Kapitel 4.5.2 bis 4.5.6 zeigen die Umsetzung dieser fünf technischen Anforderungen.

4.5.2 Fensterstruktur des Hauptdialogs

Wird die Anwendung gestartet, öffnet sich das Hauptfenster. Es wird im Folgenden als „Hauptdialog" bezeichnet und soll in insgesamt drei Bereiche unterteilt werden. Dazu werden sogenannte Frames genutzt, die von CustomTkinter bereitgestellt werden. Frames fungieren als Container, um andere GUI-Elemente logisch zu gruppieren und anzuordnen. Der Quellcode 4.3 zeigt beispielhaft die Implementierung der Frames:

```
import customtkinter as ctk

left_frame = ctk.CTkFrame(self.root, width=200, corner_radius=10)
left_frame.pack(side="left", fill="y", padx=10, pady=10)
```

Quellcode 4.3: Implementierung der Fensterstruktur in CustomTkinter

In diesem Quellcodeausschnitt wird ein Frame mit verschiedenen Parametern initialisiert. Nach demselben Prinzip wird der Hauptdialog in die Bereiche `left_frame`, für die Platzierung diverser GUI-Elemente, `right_frame_top` für die Darstellung der Karte und `right_frame_bottom` für die Darstellung der Tabelle unterteilt. Die Erstellung der Frames in CustomTkinter erfolgt durch die Klasse

`CTkFrame`. Die Anordnung der Frames im Hauptfenster wird mittels des `pack`-Managers gesteuert. [44]

4.5.3 Platzierung der GUI-Elemente

Die in Kapitel 4.5.2 erstellten Frames können anschließend mit GUI-Elementen gefüllt werden. Folgende GUI-Elemente werden mithilfe der Methode `_create_left_widgets` im `left_frame` platziert:

- Ein Button (`CTkButton`) mit der Beschriftung „Karte hochladen", welches beim Betätigen die Methode `upload_image` ausführt. Damit kann eine GeoTIFF-Datei geöffnet werden.
- Ein weiterer Button mit der Beschriftung „Gipfel finden", welches beim Betätigen die Methode `show_peaks` aufruft. Dadurch wird die Gipfelerkennung gestartet.
- Ein Schalter (`CTkSwitch`) mit der Beschriftung „3D Modus", der es dem Benutzer ermöglicht, zwischen einer 2D- und einer 3D-Ansicht der Karte umzuschalten. Der Zustand dieses Schalters wird zur Steuerung der Plotdarstellung verwendet.
- Zwei Eingabefelder (`CTkEntry`) die mit Platzhaltertexten für die aktuellen Werte der Prominenz- und Dominanz-Schwellen initialisiert sind. Diese Eingabefelder erlauben die Anpassung der Filterkriterien für die Gipfelerkennung durch den Benutzer.

In Quellcode 4.4 ist beispielhaft die Platzierung des *upload_button* dargestellt:

```
import customtkinter as ctk

upload_button = ctk.CTkButton(... command=upload_image)
upload_button.pack(pady=10, padx=20)
...
prominence_entry = ctk.CTkEntry(...)
prominence_entry.pack(pady=10, padx=20)
```

Quellcode 4.4: Implementierung des Upload-Buttons in CustomTkinter

Für die textuelle Darstellung der berechneten Gipfel wird im **`right_frame_bottom`** eine Tabelle platziert. Ein beispielhafter Ausschnitt ihrer Implementierung ist in Quellcode 4.5 dargestellt:

```
from tkinter import ttk

def _create_table(self):
    table = ttk.Treeview(self.right_frame_bottom,
    columns=("Nummer", "Pixel-Koord", "Breitengrad",
             "Längengrad", "Höhe")
    table.column("Nummer", width=40, anchor="center")
    table.column("Pixel-Koord", width=120, anchor="center")
    ...
    table.pack(expand=True, fill="both")
```

Quellcode 4.5: Erstellung der Tabelle zur Anzeige der Gipfelinformationen

Für die tabellarische Anzeige der gefundenen Gipfelkoordinaten wird in der Methode **`_create_table`** ein **`ttk.Treeview`**-Widget aus der Standard-Tkinter-Bibliothek verwendet, da **`CustomTkinter`** keine native Tabellenansicht bietet. Die Tabelle wird im **`right_frame_bottom`** platziert und verfügt über Spalten für die Nummer des Gipfels, die Pixelkoordinaten, den Breitengrad, den Längengrad und die Höhe. Die Spaltenbreiten und Ausrichtungen werden konfiguriert, um eine übersichtliche Darstellung zu gewährleisten.

4.5.4 Hochladen der DHMs

Für das Hochladen der DHMs als GeoTIFF wird die Funktion **upload_image** implementiert. Ein beispielhafter Ausschnitt ist in Quellcode 4.6 dargestellt:

```
from tkinter import filedialog
from reader import read_dem
from geo_utils import calculate_pixels_per_meter

def upload_image(self):
    file_path = filedialog.askopenfilename(filetypes=[(".tif")])
    if file_path:
        dem_data, crs, transform, (xres, yres) = read_dem(file_path)
        ...
        self.pixel_per_meter = calculate_pixels_per_meter(...)
        self._draw_plot(dem_data)
```

Quellcode 4.6: Funktion zum Hochladen der GeoTIFF-Datei

Die Methode wird aufgerufen, wenn der Benutzer auf den Knopf „Karte hochladen" klickt. Sie öffnet einen Dateiauswahldialog (**filedialog**), über den ausschließlich TIF-Dateien mit der Endung „.tif" ausgewählt werden können. Nach Auswahl einer Datei wird der Pfad gespeichert und die Funktion **read_dem** aus dem Modul **reader.py** aufgerufen, um die entsprechenden Raster- und Metadaten aus der GeoTIFF-Datei zu extrahieren. Anschließend wird die Methode **calculate_pixels_per_meter** aus **geo_utils.py** für die Weiterverarbeitung aufgerufen. Detaillierte Informationen zur Implementierung der **geo_utils.py** sind in Kapitel 4.6 gegeben.

4.5.5 Darstellung der DHMs

Um das DHM als Karte im **right_frame_top** darzustellen und zu aktualisieren, wird die Funktion **_draw_plot** verwendet. Ein beispielhafter Ausschnitt des Quellcodes ist in Quellcode 4.7 gezeigt:

```python
import numpy as np
import matplotlib.pyplot as plt

def _draw_plot(self, dem_data):
    fig = plt.figure()
    ...
    if self.dimension_switch.get() == 1: # 3D-Plot
        ax = fig.add_subplot(111, projection='3d')
        x = np.arange(dem_data.shape[1])
        y = np.arange(dem_data.shape[0])
        X, Y = np.meshgrid(x, y)
        surf = ax.plot_surface(X, Y, dem_data)
    else: # 2D-Plot
        ax = fig.add_subplot(111)
        im = ax.imshow(dem_data)
    ...
```

Quellcode 4.7: Plot-Funktion für die Kartenanzeige

Für die Funktion wird die Python-Bibliothek **`Matplotlib`** verwendet. Zunächst wird eine neue Figure erstellt, die als Leinwand dient, auf der alle Diagramme gezeichnet werden können [45]. Abhängig davon, ob der „3D Modus"-Schalter aktiviert ist, wird ein Subplot hinzugefügt. Er stellt ein einzelnes Diagramm auf der Figure dar, wobei mehrere Subplots auf einer Figure dargestellt werden können. [45] Im **2D-Modus** wird die Höhenmatrix **`dem_data`** als Heatmap dargestellt. Eine Heatmap ist eine farbcodierte Darstellung, bei der unterschiedliche Farben verschiedene Höhenwerte repräsentieren [45]. Dies geschieht mithilfe der Funktion **`ax.imshow()`**. Im **3D-Modus** werden die X- und Y-Koordinatenachsen mithilfe von **`np.meshgrid()`** erstellt. Dies ist notwendig, um die zweidimensionale Höhenmatrix in ein dreidimensionales Gitter zu transformieren. Anschließend werden die Höhenwerte als 3D-Oberfläche mit **`ax.plot_surface()`** visualisiert, was eine realistische räumliche Ansicht des Geländes ermöglicht [45]. Der erstellte Plot wird abschließend in ein Tkinter-kompatibles Widget umgewandelt und im **`right_frame_top`** des Hauptdialogs angezeigt.

4.5.6 Darstellung der berechneten Gipfel

Klickt der Benutzer auf die Schaltfläche „Gipfel finden", wird die Funktion **show_peaks** aufgerufen. Sie startet durch den Aufruf der Funktion **find_peaks** aus dem Modul **peak_analysis.py** die Berechnung der Gipfel. Kapitel 4.7 befasst sich näher mit der Implementierung dieses Moduls. Die zurückgelieferten Ergebnisse werden zunächst in der Tabelle im Hauptdialog eingetragen. Quellcode 4.8 zeigt den entsprechenden Codeausschnitt:

```
from geo_utils import convert_coordinates_to_wgs84
peaks = find_peaks(...)
for idx, (peak_xy, peak_h, prom, dom_pix) in enumerate(peaks):
    x, y = peak_xy
    z = self.dem_data[y, x] # Höhe aus DEM
    # Konvertiere Pixel-Koordinaten in zu WGS84 (Lat/Lon)
    world_x, world_y = rasterio.transform.xy(self.geo_transform, y, x)
    long, lat = convert_coordinates_to_wgs84(world_x, world_y, crs)
    ...
    # Tabelleintrag einfügen
    new_entry = (idx, f"{x}, {y}", f"{lat}", f"{long}", f"{z}")
    self.peaks_table.insert("", "end", values=new_entry)
```

Quellcode 4.8: Darstellung der berechneten Gipfel in der Tabelle

Das Array **peaks** enthält die Informationen über die gefundenen Gipfel. Es beinhaltet die Pixelkoordinaten, die Höhe, die Prominenz und die Dominanz für jeden Gipfel. Für jeden Gipfel wird nun eine Schleife durchlaufen, um die Pixelkoordinaten in geografische Koordinaten mit Längen- und Breitengrad zu überführen. Anschließend werden alle Ergebnisse in die Tabelle (**self.peaks_table**) eingetragen.

Zusätzlich werden die identifizierten Gipfel in der Kartendarstellung visualisiert. Die Darstellung erfolgt als Scatter-Plot, der die Gipfel als einzelne Punkte im Koordinatensystem einzeichnet [45]. Im 2D-Modus werden nur die x- und y-Koordinaten dargestellt. Im 3D-Modus wird zusätzlich die z-Koordinate berücksichtigt. Die Gipfel werden durch rote Dreiecke gekennzeichnet, wie in Abbildung 4.3 zu sehen ist:

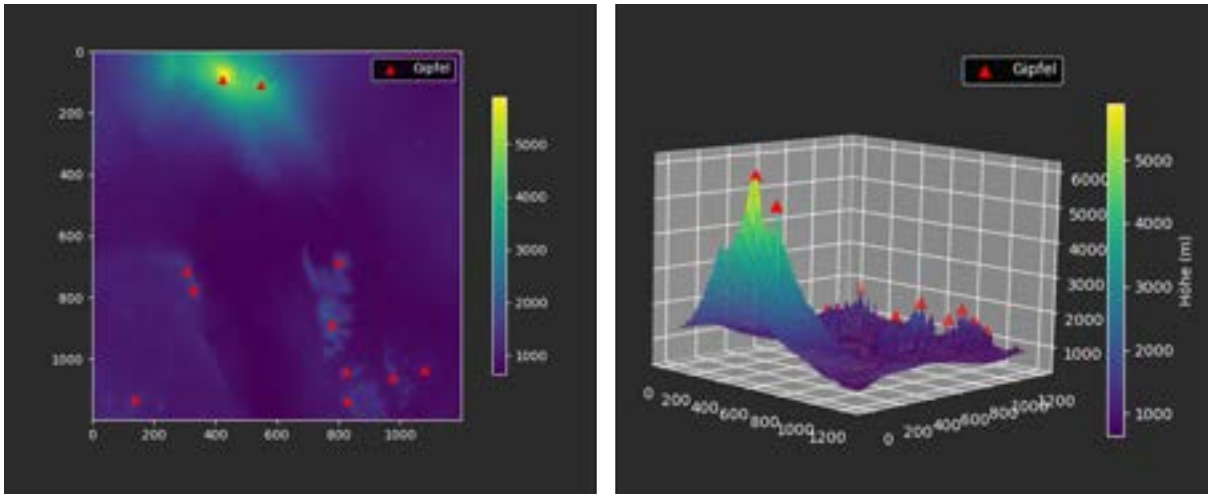

Abbildung 4.3: 2D- und 3D-Darstellung der berechneten Gipfel

Die Abbildung zeigt die zwei- und dreidimensionale Darstellung der gefundenen Gipfel. Als Beispieldaten wurde ein Ausschnitt des Kilimandscharo-Gebirges gewählt.

4.5.7 Ergebnis

Abbildung 4.4 zeigt das finale Ergebnis der GUI des Hauptdialogs:

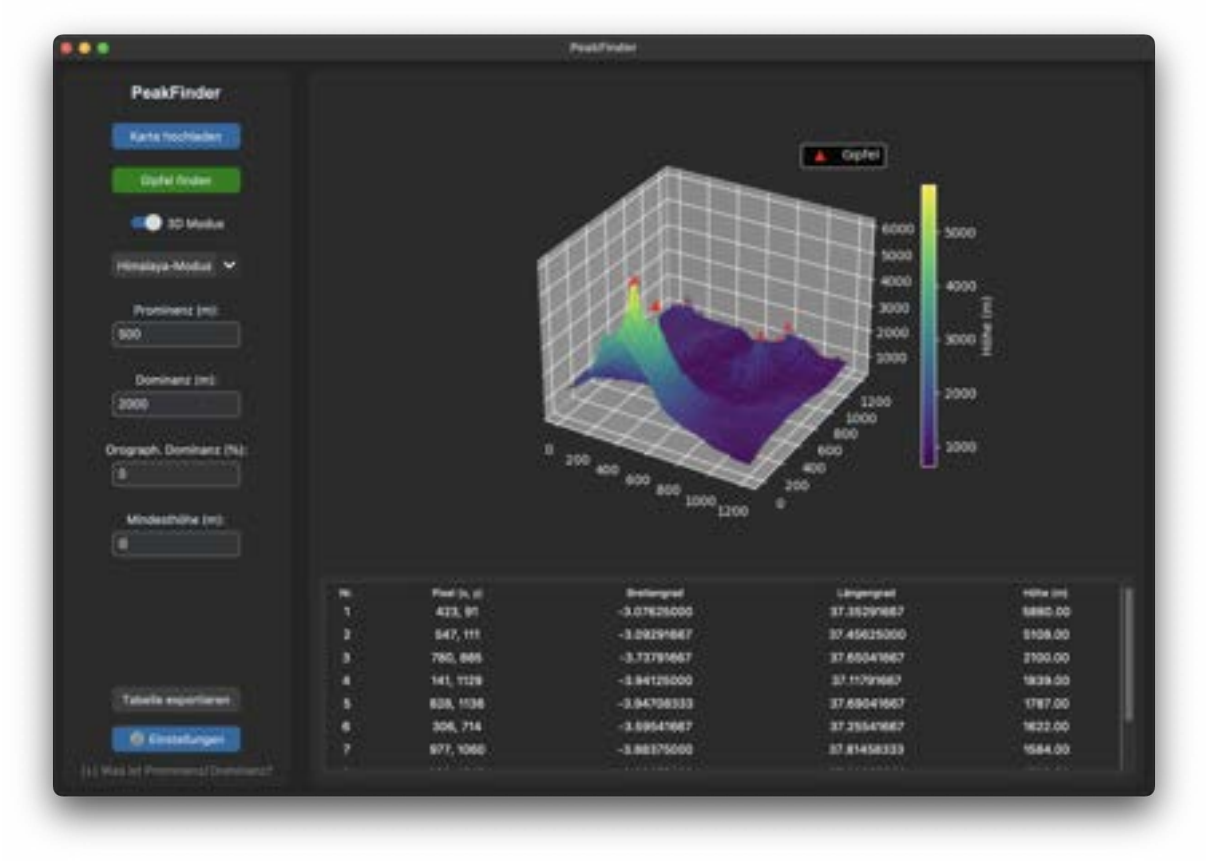

Abbildung 4.4: GUI-Fenster der Anwendung zur Gipfelerkennung

Hier sind alle für die GUI definierten technischen Anforderungen implementiert: Auf der linken Seite finden sich die Bedien- und Steuerelemenete zur Konfiguration der Gipfelberechnung, sowie die Schaltfläche zum Hochladen der DHMs. Rechts oben kann der 3D-Plot des eingegebenen DHMs inklusive der gefundenen Gipfel betrachtet werden. Rechts unten im Hauptdialog findet sich die Tabelle mit den eingetragenen Ergebnissen wieder.

4.6 Implementierung der Geofunktionen (geo_utils.py)

Dieses Modul kapselt die implementierten geowissenschaftlichen Hilfsfunktioen. Dazu zählen die Umrechnung der Gipfelkoordinaten in ein international standardisiertes Format sowie die Ermittlung der räumlichen Auflösung des Kartenmaterials. Die beiden Funktionen werden in den folgenden Kapiteln erläutert.

4.6.1 Umrechnung der Weltkoordinaten in WGS84-Koordinaten

Wie in Kapitel 2.2.6 beschrieben, enthält eine GeoTIFF-Datei eine Transformationsmatrix, um die enthaltenen Pixelkoordinaten in geografische Koordinaten, nachfolgend als „Weltkoordinaten" bezeichnet, umzuwandeln. Die sich ergebenden Weltkoordinaten können mithilfe der Funktion `convert_coordinates_to_wgs84` in WGS84-Koordinaten umgewandelt werden. Die Funktion wird mihilfe der Python-Bibliothek `pyproj` implementiert und ist in Quellcode 4.9 beispielhaft dargestellt:

```
from pyproj import CRS, Transformer

def convert_coordinates_to_wgs84(x, y, crs_system):
    source_crs = CRS.from_user_input(crs_system)
    target_crs = CRS.from_epsg(4326) # WGS84

    transformer = Transformer.from_crs(source_crs, target_crs)
    long, lat = transformer.transform(x, y)
    return long, lat
```

Quellcode 4.9: Funktion zur Konvertierung von Koordinaten nach WGS84

Die Funktion nimmt die x- und y-Koordinaten eines Punktes im ursprünglichen CRS der GeoTIFF-Datei sowie die zugehörige CRS-Bezeichnung entgegen. Zunächst wird ein `CRS`-Objekt für das Quellkoordinatensystem (`source_crs`) erstellt. Dies erfolgt mithilfe der Funktion `CRS.from_user_input()`, der die aus der GeoTIFF extrahierte CRS-Information übergeben wird. Anschließend wird

das Zielkoordinatensystem WGS84 durch die Angabe des entsprechenden EPSG-Codes (4326) als **target_crs** definiert. Um die eigentliche Koordinatentransformation durchzuführen, wird ein **Transformer**-Objekt initialisiert. Dies geschieht mit der Funktion **from_crs()**, der das Quell- und Ziel-CRS übergeben wird. Die eigentliche Umwandlung der Koordinaten vom Quell- zum Zielsystem erfolgt durch den Aufruf der Methode **transform(x, y)**. Diese Methode gibt die transformierten Koordinaten zurück [37], die in diesem Fall den Längengrad (**long**) und den Breitengrad (**lat**) im WGS84-System darstellen. Durch die Verwendung von **pyproj** wird eine genaue Transformation der GeoTIFF-Koordinaten in die WGS84-Koordinaten gewährleistet, was die Vergleichbarkeit der extrahierten Gipfelkoordinaten erleichtert.

4.6.2 Umrechnung der Auflösung in Pixel pro Meter

Für die korrekte Anwendung des Dominanzkriteriums ist es notwendig, die räumliche Auflösung der GeoTIFF-Datei von ihrer ursprünglichen Einheit (Grad oder Meter pro Pixel) in Pixel pro Meter umzurechnen. Die Funktion **calculate_pixels_per_meter** führt diese Umrechnung unter Berücksichtigung des jeweiligen CRS durch. Quellcode 4.10 zeigt die Funktion:

```
from pyproj import CRS, Geod

def calculate_pixels_per_meter(crs, px_scale, top_left_x, top_left_y):
    crs = CRS.from_user_input(crs)
    pixel_scale_x, pixel_scale_y = px_scale

    if crs.is_geographic: # Einheit ist Grad
        geod = Geod(ellps="WGS84") # Standard-Ellipsoid

        # Horizontal:
        lon1 = top_left_x
        lon2 = top_left_x + pixel_scale_x
        lat = top_left_y
        _, _, dist_x = geod.inv(lon1, lat, lon2, lat)
        ...
```

Quellcode 4.10: Funktion zur Umrechnung der Kartenauflösung in Pixel pro Meter

Die Funktion nimmt das CRS-System der GeoTIFF-Datei, den Pixelmaßstab und die Koordinaten der oberen linken Ecke des Rasters als Eingabeparameter entgegen. Für die Berechnung der Pixel pro Meter wird zunächst geprüft, ob das CRS geografisch (Einheit in Grad) oder projiziert (Einheit in Metern) ist: Wenn das CRS geografisch ist, müssen die Winkelmaße in metrische Distanzen umgerechnet werden, dafür wird die Funktion `geod.inv()` verwendet. Diese Funktion berechnet die direkte geodätische Distanz zwischen zwei Punkten, die durch ihre Längen- und Breitengrade definiert sind [37]. Die Umrechnung wird für die horizontale (x) und vertikale (y) Richtung ausgeführt:

- **Horizontale Distanz (`dist_x`):** Um die horizontale Distanz zu bestimmen, werden zwei Punkte auf demselben Breitengrad verglichen. Der erste Punkt ist die Zielkoordinate, und der zweite Punkt ist diese Koordinate, verschoben um die Pixelbreite in Grad. Mit `geod.inv()` wird dann mit den zwei Punkten die Distanz `dist_x` in Metern berechnet.
- **Vertikale Distanz (`dist_y`):** Die vertikale Distanz wird analog zur horizontalen Distanz berechnet, wobei hier zwei Punkte auf demselben Längengrad betrachtet werden.

Wenn das CRS projiziert ist, ist die Einheit des Pixelmaßstabs bereits metrisch. In diesem Fall können die Werte direkt übernommen werden. Nachdem die Distanzen `dist_x` und `dist_y` ermittelt wurden, erfolgt die Berechnung der Pixel pro Meter durch Bildung des Kehrwerts. Für die Anwendungen im Rahmen dieses Programms wird angenommen, dass die Pixel quadratisch sind, d.h. `dist_x` und `dist_y` sind gleich. Daher wird nur einer der beiden Werte für die Umrechnung der Dominanz von Metern in Pixel verwendet.

4.7 Implementierung der Gipfelfindung (peak_analysis.py)

Dieses Kapitel beschreibt die angewandten Methoden zur Bestimmung potenzieller Gipfel, zur Berechnung ihrer Prominenz, Dominanz und orografischer Dominanz sowie die jeweiligen Komplexitätsanalysen und Performance-Aspekte der implementierten Algorithmen der Datei `peak_analyis.py`.

4.7.1 Überblick

Die Gipfelerkennung verarbeitet das Höhenmodell schrittweise. Die Hauptfunktion der `peak_analyis.py`, ist die Kombination der Berechnung der potenziellen Gipfel mit einer anschließenden Filterung. Diese Filterung basiert auf den in Kapitel 2.3 definierten Kriterien:

1. **Lokale Maxima finden:** Zuerst wird `find_local_maxima` aufgerufen, um die Pixelkoordinaten aller potenziellen Gipfel zu ermitteln.
2. **Filterung nach Prominenz:** Anschließend bestimmt `calculate_prominent_peaks` die Prominenz jedes Kandidatengipfels. Gipfel, deren Prominenz unter einem definierten Wert liegt, werden verworfen.
3. **Filterung nach Dominanz:** Danach berechnet `calculate_dominance_distance` die Dominanz. Nur Gipfel, deren Dominanz einen bestimmten Wert überschreitet, bleiben erhalten.
4. **Filterung nach orografischer Dominanz:** Im Anschluss erfolgt die Berechnung der orografischen Dominanz mittels `calculate_orographic_dominance`. Gipfel unterhalb eines festgelegten Wertes werden ausgeschlossen.
5. **Filterung nach Mindesthöhe:** Gipfel unter einer Mindesthöhe werden herausgefiltert.

4.7.2 Bestimmung der potenziellen Gipfel

Bevor prominente Gipfel identifiziert werden können, muss zunächst bestimmt werden, welche Punkte im digitalen Höhenmodell überhaupt als potenzielle Gipfelkandidateinfragege kommen. Dieser Schritt stellt die Grundlage für die spätere Prominenz- und Dominanzberechnung dar.

Eliminierung von Randpunkten Um zu verhindern, dass Punkte am Rand des digitalen Höhenmodells fälschlicherweise als Gipfel identifiziert werden, und um die potenziellen Gipfelkandidaten zu bestimmen, werden zwei aufeinanderfolgende Schritte implementiert: Das Setzen der Bildränder auf Null und die anschließende Suche nach lokalen Maxima im verarbeiteten DHM. Die Funktion `set_image_borders_to_zero` dient dazu, die Pixelwerte an den äußeren Rändern des DHM auf Null zu setzen. Dies stellt sicher, dass diese Randbereiche bei der nachfolgenden Suche nach lokalen Maxima nicht berücksichtigt werden. Die Funktion nimmt das Höhenmodell und eine Breite als Eingabe. Alle Pixel innerhalb dieser Breite vom oberen, unteren, linken und rechten Rand des Bildes werden auf den Wert Null gesetzt.

Berechnung von lokalen Maxima Um die potenziellen Gipfelpositionen im DHM zu identifizieren, wird ein Algorithmus zur Findung lokaler Maxima angewendet. Dieser basiert auf der Nutzung des Maximum-Filters aus der `scipy.ndimage`-Bibliothek. Die Funktion `find_local_maxima`, die den Algorithmus implementiert ist in Quellcode 4.11 dargestellt:

```python
import numpy as np
from scipy.ndimage import maximum_filter

def find_local_maxima(img_data, border_width=2):
    # Ränder des Bildes ausschließen
    img_data = set_image_borders_to_zero(img_data, width=border_width)

    # Wende Maxima-Filter an
    max_out = maximum_filter(img_data, size=7)
    local_max = np.zeros((img_data.shape))
    local_max[max_out == img_data] = 1

    # Finde Koordinaten von Maxima -> Liste von Maxima
    local_max_list = np.argwhere(local_max == 1)
    return local_max_list
```

Quellcode 4.11: Algorithmus zur Findung lokaler Maxima

Die Funktion nimmt das Höhenmodell (`img_data`) und eine Breite für den Randbereich (`border_width`) entgegen und führt folgende Schritte aus:

1. **Ausschließen der Bildränder:** Zunächst wird die Funktion `set_image_borders_to_zero` auf das Eingabebild angewendet, um die Werte an den Rändern zu eliminieren und somit potenzielle Randgipfel von der weiteren Analyse auszuschließen.

2. **Anwenden des Maximum-Filters:** Anschließend wird der `maximum_filter` auf das bearbeitete Höhenmodell angewendet. Dieser Filter ersetzt jeden Pixelwert im Bild durch das Maximum innerhalb einer definierten Nachbarschaft (z.B. 7x7 Pixel) [39]. Das Ergebnis (`max_out`) ist eine neue Matrix, in der jeder Wert dem lokalen Maximum in seiner Umgebung entspricht. Ein lokales Maximum liegt vor, wenn der Wert eines Pixels im ursprünglichen Höhenmodell gleich der Ergebnismatrix (`max_out`) ist.

3. **Extraktion der Koordinaten:** Schließlich werden die Koordinaten der lokalen Maxima extrahiert. Die Funktion `np.argwhere()` gibt eine Liste von Koordinatenpaaren zurück [38], wobei jedes Paar die y- und x-Koordinate eines lokalen Maximums repräsentiert.

Die Zeitkomplexität der Funktion `find_local_maxima` wird durch die Gesamtanzahl der Pixel im Eingabebild `img_data` bestimmt. Sie beträgt $O(N)$ oder äquivalent $O(H \cdot B)$, wobei N die Anzahl der Pixel, H die Höhe und B die Breite des Bildes repräsentiert. Dies impliziert eine lineare Skalierung der Laufzeit mit der Bildgröße. Die wesentlichen Komponenten, die zu dieser Komplexität beitragen, sind wie folgt zu bewerten:

- **`set_image_borders_to_zero`:** Diese Funktion modifiziert die Pixel an den Rändern des Bildes. Obwohl sie auf mehrere Zeilen bzw. Spalten zugreift, betrifft die Operation nur einen relativ kleinen Teil des Gesamtbildes. Der Beitrag zur Komplexität ist daher im Vergleich zu Operationen, die das gesamte Bild durchlaufen, vernachlässigbar und unterschreitet $O(N)$.
- **`max_out = maximum_filter`:** Die Anwendung des `maximum_filter` involviert die Verarbeitung jedes Pixels im Bild. Da die Filtergröße (hier $7 \cdot 7$) konstant ist, ist die benötigte Rechenzeit direkt proportional zur Gesamtzahl der Pixel, was einer Komplexität von $O(N)$ entspricht.
- **`local_max[max_out == img_data] = 1`:** Der elementweise Vergleich zwischen den zwei Arrays `max_out` und `img_data`, die beide die Größe $H \cdot B$ besitzen, resultiert in einer Komplexität von $O(N)$.

Zusammenfassend lässt sich feststellen, dass alle dominanten Schritte innerhalb der Funktion eine Komplexität von $O(N)$ aufweisen. Daher ist die Gesamtzeitkomplexität der Funktion `find_local_maxima` linear in Bezug auf die Anzahl der Pixel des Eingabebildes. Auch ist anzumerken, dass der `maximum_filter` von der Scipy-Bibliothek in C geschrieben ist, weshalb die Berechnung lokaler Maxima sehr schnell erfolgen kann.

4.7.3 Berechnung der Prominenz

Der erste Schritt bei der Ermittlung der Prominenz ist die Bestimmung des nächsthöheren Gipfels. Wie in Kapitel 3.3 beschrieben, folgt ein zweistufiger Ansatz: Der Bresenham-Algorithmus gefolgt von dem Max-Min-Dijkstra-Algorithmus.

Die Funktion `calculate_prominent_peaks` berechnet die Prominenz der Gipfel und filtert diejenigen heraus, deren Prominenz einen gegebenen Schwellenwert überschreitet. Quellcode 4.12 zeigt die Implementierung dieser Funktion:

```python
import numpy as np

def calculate_prominent_peaks(candidate_peaks_xy, height_map,
                              prom_threshold):
    # Nearest-Higher Gipfel finden für jeden Gipfel
    nearest = compute_nearest_higher(coords, heights)
    prominent_peaks = []
    for i in range(len(coords)):
        x, y = coords[i]
        h = heights[i]
        j = nearest[i] # Position vom nächsthöheren Gipfel
        if j == -1: # Höchster Peak
            prominent_peaks.append(((x, y), int(h), int(h)))
            continue
        # Pfad und Sattelpunkt erst mit Bresenham-Approximation
        path = get_path_between_points((x, y), tuple(coords[j]))
        saddle_h = min(height_map[yy, xx] for xx, yy in path)
        prom = h - saddle_h
        if prom >= prom_threshold:
            # Feine Berechnung des Sattels mit Maxmin-Dijkstra
            saddle_h = get_maxmin_saddle(height_map, (x, y), ...)
            prom = h - saddle_h
            if prom >= prom_threshold:
               prominent_peaks.append(((x, y), int(h), int(prom)))
    return prominent_peaks
```

Quellcode 4.12: Berechnung der Prominenz von Gipfeln

Zunächst werden die Eingabekoordinaten **`candidate_peaks_xy`** und das Höhenmodell **`height_map`** in NumPy-Arrays umgewandelt. Die Höhenwerte der Gipfel werden aus dem Höhenmodell extrahiert. Die Gipfel werden dann in absteigender Reihenfolge ihrer Höhe sortiert. Dies ist eine übliche Optimierung, da höhere Gipfel tendenziell eine größere Prominenz haben und somit schneller gefunden werden können. Für jeden Gipfel i wird der nächsthöhere Nachbar j aus dem Array **`nearest`** abgerufen. Wenn **`nearest[i]`** gleich -1 ist, bedeutet dies, dass der aktuelle Gipfel der höchste Punkt im betrachteten Gebiet ist und seine Prominenz gleich seiner Höhe ist. Andernfalls wird der Pfad zwischen dem Gipfel i und seinem nächsthöheren Nachbarn j mit der Funktion **`get_path_between_points`**

berechnet. Der niedrigste Punkt auf diesem Pfad, der sogenannte Sattelpunkt, wird bestimmt. Die Prominenz des Gipfels i ist dann die Höhendifferenz zwischen dem Gipfel und dem Sattelpunkt. Gipfel, deren Prominenz den gegebenen Schwellenwert **`prominence_threshold`** überschreitet, werden nochmal mit **`get_maxmin_saddle`** gefiltert. Dabei wird die Prominenz ausgerechnet und dann nochmal dem Schwellenwert verglichen. Am Ende werden die Gipfel mit ihren Koordinaten, ihrer Höhe und ihrer Prominenz in der Liste **`prominent_peaks`** gespeichert.

Bestimmung des nächsthöheren Nachbarn Die Funktion **`compute_nearest_higher`** nimmt die Koordinaten der potenziellen Gipfel (**`coords`**) und deren Höhenwerte (**`heights`**) als Eingabe und bestimmt den zugehörigen nächsthöheren Nachbargipfel. Quellcode 4.13 stellt diese Funktion dar:

```
import numpy as np
from numba import njit

@njit
def compute_nearest_higher(coords, heights):
    ...
    for i in range(n):
        ...
        for j in range(n):
            hj = heights[j]
            if hj > hi: # höheren Gipfel gefunden
                dx = xi - coords[j, 0]
                dy = yi - coords[j, 1]
                d = np.hypot(dx, dy) # euklidische Distanz
                if d < min_d: # näheren Gipfel gefunden
                    min_d = d
                    best = j
        nearest[i] = best
    return nearest
```

Quellcode 4.13: Berechnung des nächsthöheren Nachbarn

Für jeden Punkt i sucht die Funktion denjenigen Punkt j, der eine größere Höhe hat und die geringste Distanz zu i aufweist. Die Distanz wird dabei mittels der euklidischen Distanz zwischen den Pixelkoordinaten berechnet [27]:

$$d = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

1. **Schleife über jeden lokalen Maxima „i":** Es wird ein Array `nearest` initialisiert, in dem für jeden Punkt i der Index des nächsthöheren Nachbarn gespeichert wird. Zunächst wird es mit dem Wert -1 gefüllt, was bedeutet, dass noch kein höherer Nachbar gefunden wurde.

2. **Schleife über alle anderen Maxima „j":** Für jeden Punkt i wird eine Schleife über alle anderen Punkte j durchgeführt. Ist der Punkt j höher als i, wird die euklidische Distanz berechnet. Dafür wird die Numpy-Funktion `hypot` verwendet.

3. **Bestimmung des nächsthöheren Gipfel:** Die Funktion speichert jeweils denjenigen höheren Punkt, der die geringste Distanz zu i hat. Wurde kein höherer Punkt gefunden (z.B. wenn Punkt i der höchste von allen ist), bleibt der Eintrag gleich -1.

Die Funktion gibt ein Array `nearest` zurück, wobei `nearest[i]` den Index des nächsthöheren Nachbarn von Punkt i enthält. Wenn ein Punkt keinen höheren Nachbarn hat, wird der entsprechende Eintrag in `nearest` auf -1 gesetzt.

Die Zeitkomplexität der Funktion `compute_nearest_higher` ist quadratisch, d.h., $O(N^2)$, wobei N die Anzahl der als Eingabe übergebenen Koordinaten (bzw. der gefundenen lokalen Maxima) darstellt. Diese Komplexität ergibt sich aus der Struktur der verschachtelten Schleifen und der darin ausgeführten Operationen:

- **Äußere Schleife (`for i in range(n)`):** Diese Schleife iteriert N-mal, einmal für jede Koordinate in der Eingabeliste.

- **Innere Schleife (`for j in range(n)`):** Diese Schleife ist innerhalb der äußeren Schleife positioniert und wird ebenfalls N-mal für jede Iteration der äußeren Schleife durchlaufen. Dies führt zu insgesamt $N \cdot N$ Vergleichen zwischen den Koordinaten.

- **Operationen innerhalb der inneren Schleife: `d = np.hypot(dx, dy)`:** Die Berechnung der Hypotenuse zur Bestimmung der euklidischen Distanz zwischen zwei Punkten ist eine arithmetische Operation konstanter Zeit, $O(1)$.
  Vergleiche und Zuweisungen (`if d < min_d: min_d = d; best = j`): Diese Operationen sind ebenfalls atomar und haben jeweils eine konstante Zeitkomplexität von $O(1)$.

Da jede Iteration der inneren Schleife Operationen mit einer Komplexität von $O(1)$ ausführt und die innere Schleife selbst N-mal durchlaufen wird, beträgt die Komplexität jeder Iteration der äußeren Schleife $N \cdot O(1) = O(N)$. Da die äußere Schleife ebenfalls N-mal durchlaufen wird, addiert sich die Gesamtkomplexität der verschachtelten Schleifen zu $N \cdot O(N) = O(N^2)$. Aufgrund der hohen quadratischen Komplexität von $O(N^2)$ für die Berechnung des nächsthöheren Gipfels wird in der Implementierung Numba eingesetzt. Durch das Anbringen des Dekorators `@njit` an die Funktion wird diese in optimierten Maschinencode kompiliert. Dies führt zu einer signifikanten Beschleunigung der Ausführung, auch wenn die zugrundeliegende algorithmische Skalierung der Laufzeit mit der Eingabegröße (N^2) unverändert bleibt. Die resultierende Performancesteigerung ist ein kritischer Aspekt für die Praktikabilität des Programmes und wird in 4.7.3 evaluiert.

Berechnung mit Bresenham-Algorithmus Die Funktion `get_path_between_points` verwendet den Bresenham-Algorithmus, der in der Bibliothek `scikit-image` implementiert ist, um eine Approximation der Prominenz zu berechnen. Quellcode 4.14 zeigt die Implementierung davon.

```
from skimage.draw import line

def get_path_between_points(p1, p2):
    rr, cc = line(p1[1], p1[0], p2[1], p2[0])
    return list(zip(cc, rr)) # -> Liste von (x,y) Tupeln
```

Quellcode 4.14: Berechnung des Pfades zwischen zwei Punkten mit Bresenham-Algorithmus

Die Funktion `line` erwartet die Koordinaten in der Reihenfolge (Zeile, Spalte), während die Gipfelkoordinaten als (x, y) = (Spalte, Zeile) vorliegen [40]. Daher

ist eine entsprechende Anpassung der Koordinatenreihenfolge notwendig. Die Funktion gibt eine Liste von (x, y)-Tupeln zurück, die den approximierten Pfad zwischen den beiden Punkten darstellen.

Die Zeitkomplexität der Funktion `get_path_between_points` wird maßgeblich von der Anzahl der Pixel bestimmt, die von der Funktion `skimage.draw.line` generiert werden. Diese Anzahl, im Folgenden mit k bezeichnet, ist direkt proportional zur Länge der Linie in Pixeln. Die Analyse der Funktion erfolgt Schritt für Schritt:

- **`rr, cc = line(p1[1], p1[0], p2[1], p2[0])`:** Die Funktion `line` ist für die Generierung der Zeilen- (`rr`) und Spaltenkoordinaten (`cc`) der Pixel zuständig, welche eine gerade Linie zwischen den Punkten `p1` und `p2` im Raster bilden. Algorithmen zur Linienrasterung, wie der zugrundeliegende Bresenham-Algorithmus, weisen eine Komplexität auf, die linear von der Anzahl der erzeugten Punkte abhängt. Somit beträgt die Komplexität dieses Schrittes $O(N)$. Dabei ist $N = p_2 - p_1$
- **`list(zip(cc, rr))`:** Diese Operation wandelt die Paare von Zeilen- und Spaltenkoordinaten in eine Liste von Tupeln um. Hierbei werden N Tupel erstellt und der Liste hinzugefügt. Auch dieser Prozess skaliert linear mit der Anzahl der Punkte und hat somit eine Komplexität von $O(N)$.

Die dominanten Operationen innerhalb der Funktion, sowohl die Generierung der Linienkoordinaten als auch deren Umwandlung in eine Liste von Tupeln, sind linear von der Anzahl der Punkte N auf der Linie abhängig. Daher ist die Gesamtzeitkomplexität der Funktion `get_path_between_points` linear, d.h., $O(N)$. Dadurch ist diese Berechnung schnell.

Berechnung mit dem Max-Min-Dijkstra-Algorithmus Der Algorithmus findet den „Maxmin-Pfad" – der Pfad, auf dem der minimale Höhenwert des gesamten Pfades maximiert wird (siehe 3.3). Die Funktion `maxmin_path_saddle` nimmt die DHM, die Startkoordinaten des eines Gipfels (`start`) und die Zielkoordinaten des nächsthöheren Gipfels (`end`) als Eingabe entgegen und gibt die Höhe des Sattelpunktes zurück. Die Implementierung ist in Quellcode 4.15 dargestellt.

```python
import heapq

@njit
def get_maxmin_saddle(height_map, start, end):
    rows, cols = height_map.shape
    sx, sy = start
    ex, ey = end

    best = np.full((rows, cols), -np.inf, dtype=np.float64)
    best[sy, sx] = float(height_map[sy, sx])
    pq = [(-best[sy, sx], sx, sy)] # (-Sattelhöhe, x, y)

    while pq:
        cur_min_neg, x, y = heapq.heappop(pq)
        cur_min = -float(cur_min_neg)

        if (x, y) == (ex, ey): # Wenn am Ziel, Wert zurückgeben
            return cur_min

        # 4-Nachbarn
        for dx, dy in ((1,0), (-1,0), (0,1), (0,-1)):
            nx, ny = x + dx, y + dy
            if 0 <= nx < cols and 0 <= ny < rows:
                neigh_h = float(height_map[ny, nx])
                saddle = min(cur_min, neigh_h)
                if saddle > best[ny, nx]:
                    best[ny, nx] = saddle
                    heapq.heappush(pq, (-saddle, nx, ny))

    return best[ey, ex]  # Falls Ziel nie erreicht wurde
```

Quellcode 4.15: Berechnung der Prominenz mit dem Max-Min-Dijkstra-Algorithmus

Der modifizierte Dijkstra-Algorithmus funktioniert in den folgenden Schritten:

1. **Initialisierung:** Ein 2D-Array `best` wird initialisiert. Alle Werte werden auf $(-\infty)$ gesetzt, außer dem Startpunkt, dessen Wert auf seine eigene Höhe gesetzt wird. Zusätzlich wird eine Prioritätswarteschlange (`pq`, implementiert

mit `heapq`) erstellt. Die Sattelhöhe wird negativ gespeichert, da `heapq` standardmäßig einen Min-Heap ist [41]. So wird sichergestellt, dass Knoten mit der höchsten Sattelhöhe (dem kleinsten negativen Wert) zuerst verarbeitet werden. Der Startpunkt wird mit seiner negativen Höhe in die Warteschlange eingefügt.

2. **Pfadsuche und Aktualisierung:** Für jeden der vier direkten Nachbarn des aktuellen Knotens wird iterativ folgende Schritte durchgeführt:
   - Es wird geprüft, ob der Nachbar innerhalb der Grenzen des Höhenmodells liegt und die Höhe des Nachbarn (`neigh_h`) wird ermittelt.
   - Der neue mögliche Sattelpunkt für den Pfad zum Nachbarn (`saddle`) wird berechnet. Dies ist das Minimum aus der aktuellen minimalen Höhe des Pfades (`cur_min`) und der Höhe des Nachbarn (`neigh_h`).
   - Wenn diese neu berechnete Höhe (`saddle`) größer ist als der aktuell höchste Wert für die Sattelhöhe, wird `best[ny, nx]` aktualisiert.
3. **Rückgabe:** Der Algorithmus terminiert beim Erreichen des Ziels.

Dieser modifizierte Dijkstra-Algorithmus gewährleistet, dass der tatsächliche Sattelpunkt zwischen zwei Gipfeln gefunden wird.

Die Zeitkomplexität der Funktion `maxmin_path_saddle`, die einen Maxmin-Pfad auf einer gegebenen Höhenkarte (`height_map`) berechnet, beträgt $O(N \log N)$. Hierbei repräsentiert N die Gesamtzahl der Pixel bzw. Zellen in der `height_map`. Diese Komplexität ist charakteristisch für Dijkstra-ähnliche Algorithmen, die Prioritätswarteschlangen (Heaps) verwenden. Die detaillierte Aufschlüsselung der Komponenten, die zu dieser Komplexität beitragen, ist wie folgt:

- **`while pq`: Schleife:** Diese Hauptschleife wird so lange ausgeführt, wie sich Elemente in der Prioritätswarteschlange `pq` befinden. Im ungünstigsten Fall kann jede Zelle der `height_map` einmal zur Prioritätswarteschlange hinzugefügt und einmal daraus entfernt werden.
- **Operationen innerhalb der `while`-Schleife: `heapq.heappop`:** Das Entfernen des Elements mit der höchsten Priorität aus einem Heap mit k Elementen erfordert eine Zeitkomplexität von $O(\log k)$. Da die Prioritätswarteschlange bis zu N Elemente enthalten kann, beträgt die Komplexität dieser Operation

$O(\log N)$. Diese Operation wird maximal N-mal ausgeführt (einmal für jede "besuchteSZelle). Die kumulierten Kosten für alle `heappop`-Operationen belaufen sich somit auf $N \cdot O(\log N) = O(N \log N)$. **`heapq.heappush`:** Für jede aus der Warteschlange entnommene Zelle werden deren bis zu vier Nachbarn überprüft. Dies stellt eine konstante Anzahl von Operationen pro Zelle dar. Wenn der Pfad zu einem Nachbarn verbessert wird, wird der Nachbar der Prioritätswarteschlange hinzugefügt. Das Hinzufügen eines Elements zu einem Heap mit bis zu N Elementen kostet $O(\log N)$. Diese `heappush`-Operation kann für jede „Kante" im impliziten Graphen betrachtet werden. In diesem Graphen gibt es $O(N)$ Kanten (ungefähr $4N$). Die Gesamtkosten für alle `heappush`-Operationen sind somit $O(E \log N)$, wobei E die Anzahl der Kanten ist. Da $E = O(N)$, ergibt sich hieraus ebenfalls eine Komplexität von $O(N \log N)$.

Die dominierenden Terme in der Zeitkomplexität sind die kumulierten Kosten für die `heappop`-Operationen und die `heappush`-Operationen. Beide summieren sich zu $O(N \log N)$. Daher ist die Gesamtzeitkomplexität der Funktion `maxmin_path_saddle` $O(N \log N)$. Um die Ausführungsgeschwindigkeit dieser rechenintensiven Funktion zu optimieren, wird hier ebenfalls der `@njit`-Dekorator von Numba verwendet. Obwohl die `heapq`-Funktionen selbst bereits in C implementiert und somit sehr schnell sind, kann Numba effektiv mit Python-Listen (die von `heapq`) interagieren und die Koordination dieser Aufrufe optimieren.

Evaluierung der Performance Um die Effektivität der Numba-Optimierung in der Prominenzberechnung zu evaluieren, wurde ein spezifischer Test durchgeführt. Dieser Test konzentriert sich auf die Funktion `calculate_prominence_peaks` und verwendet synthetische Testdaten in Form eines 2D-Arrays mit zufälligen Höhenwerten. Die Berechnung wird dabei einmal mit und einmal ohne den `@njit`-Dekorator von Numba ausgeführt. Die Berechnungen wurden mit einem Macbook Air ausgestattet, mit einem M2 Apple-Sillicon Chip und 8 GB Arbeitsspeicher ausgeführt.

Da die Vorfilterung mittels Bresenham bei zufälligen Daten die Anzahl lokaler Maxima kaum reduziert und der Dijkstra-Algorithmus in diesem Kontext nicht relevant ist, wird die Berechnung mit Dijkstra ausgelassen. Damit ermöglicht dieser Ansatz einen direkten und klaren Vergleich der Numba-Optimierung. Konkret liegt

der Fokus auf der rechenintensiven Funktion `compute_nearest_higher`, die eine quadratische Zeitkomplexität von $O(N^2)$ aufweist. Die Ergebnisse der Performance-Messungen sind in Abbildung 4.5 dargestellt:

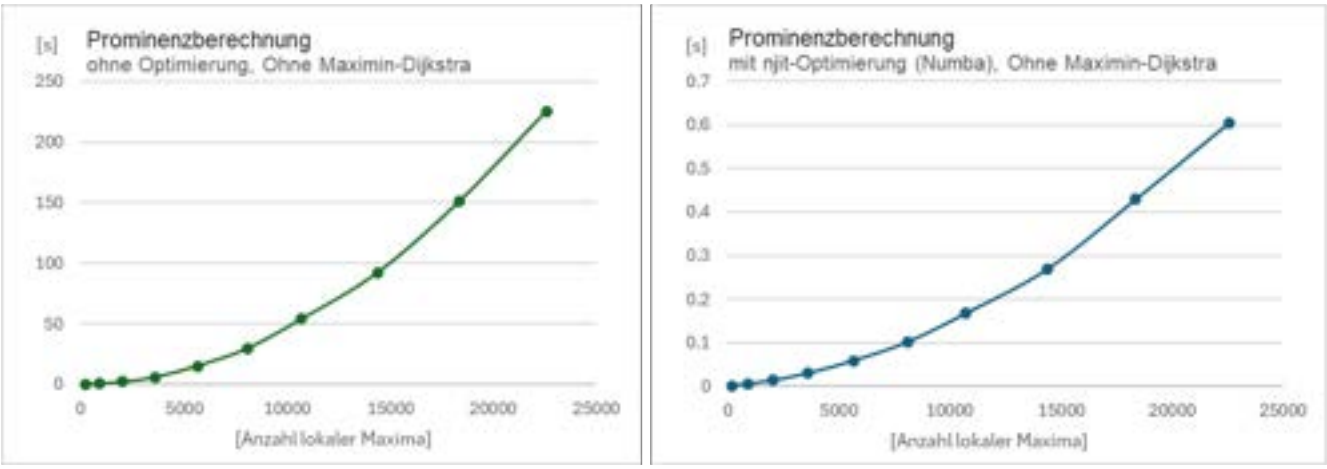

Abbildung 4.5: Vergleich der Dauer der Prominenzberechnung mit und ohne Optimierung

Wie erwartet, zeigen beide Kurven einen quadratischen Anstieg, was die dominierende Komplexität von $O(N^2)$ der Funktion `compute_nearest_higher` bestätigt. Dennoch ist ein erheblicher Unterschied in der Berechnungsdauer zu beobachten. So ist bei einer Testdatei, die 22.589 lokale Maxima beinhaltet, die Numba-optimierte Prominenzberechnung etwa 373-mal schneller als die Standardimplementierung (0,60496s gegenüber 225,45982s). Diese hohe Beschleunigung unterstreicht den entscheidenden Vorteil der Just-In-Time-Kompilierung durch Numba für rechenintensive Operationen.

4.7.4 Berechnung der Dominanz

Weitere wichtige topografische Merkmale eines Gipfels ist seine Dominanz und seine orografische Dominanz. Die Dominanz beschreibt, wie weit ein Gipfel vom nächsthöheren Punkt entfernt liegt, während die orografische Dominanz das Verhältnis der Schartenhöhe eines Gipfels zu seiner Höhe beschreibt. Im folgenden Abschnitt wird erläutert, wie die Dominanzdistanz und die orografische Dominanz berechnet und algorithmisch ermittelt wird.

Die Funktion **`calculate_dominance_distance`** ist für die Berechnung der Dominanz zuständig. Dabei wird analog zu **`compute_nearest_higher`** die euklidische Distanz zwischen zwei Punkten betrachtet.

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Quellcode 4.16 zeigt die Implementierung der Dominanzberechnung:

```
import numpy as np
from scipy.ndimage import distance_transform_edt

def calculate_dominance_distance(peak_xy, height_map):
    x, y = peak_xy
    h0 = height_map[y, x]

    mask = (height_map < h0)
    mask[y, x] = True   # den eigenen Gipfel entfernen
    dist_map = distance_transform_edt(mask)
    return dist_map[y, x]
```

Quellcode 4.16: Berechnung der Dominanzdistanz

Die Dominanzbestimmung funktioniert in den folgenden Schritten:

1. **Höhe abrufen:** Die Funktion erhält die (x, y)-Koordinaten des zu analysierenden Gipfels (**`peak_xy`**) sowie den digtalen Höhenmodell. Zunächst wird der Höhenwert des Gipfels (**`h0`**) aus der DHM extrahiert.

2. **Maske erstellen:** Eine boolesche Maske wird generiert, in der alle Pixel der **`height_map`**, deren Höhenwert kleiner als der des Gipfels (**`h0`**) ist, den Wert **`True`** erhalten. Die **`False`**-Werte der Maske repräsentieren damit die höheren Punkte.

3. **Distanztransformation:** Die Funktion **`distance_transform_edt`** wird auf die erstellte binäre Maske angewendet. Dabei handelt es sich um eine euklidische Distanztransformation. Diese Transformation berechnet für jedes **`True`**-Pixel in der Maske den euklidischen Abstand zum nächstgelegenen **`False`**-Pixel [39]. Diese Abstände werden in **`dist_map`** gespeichert.

4. **Dominanz-Distanz zurückgeben:** Die `dist_map` enthält für jedes Pixel den Abstand zum nächstgelegenen „höheren" Punkt. Um die Dominanz zu erhalten, gibt die Funktion gibt den Wert der `dist_map` bei den Koordinaten des Gipfels (`y, x`) zurück.

Die Zeitkomplexität der Funktion `calculate_dominance_distance` wird maßgeblich durch die Operation `distance_transform_edt` bestimmt, da diese den rechenintensivsten Teil darstellt. Die Komplexität der einzelnen Schritte ist wie folgt:

- **`mask = (height_map < h0)`:** Diese Operation erstellt eine boolesche Maske, indem sie jedes Element der `height_map` mit `h0` vergleicht. Bei einer `height_map` der Größe $H \cdot B$ Pixel (wobei H die Höhe und B die Breite ist) müssen $H \cdot B$ Vergleiche durchgeführt werden. Die Komplexität dieses Schrittes ist somit $O(H \cdot B)$ oder $O(N)$, wobei N die Gesamtzahl der Pixel ist.

- **`dist_map = distance_transform_edt(mask)`:** Dies ist der aufwendigste Teil. Die euklidische Distanztransformation ist ein hochoptimierter Algorithmus in SciPy. Für 2D-Bilder (wie eine Höhenkarte) wird die Komplexität dieser Operation als $O(N)$ oder $O(H \cdot B)$ beschrieben. Dies liegt daran, dass moderne Algorithmen für die Distanztransformation, wie sie in SciPy implementiert sind, die Transformation in einer konstanten Anzahl von Durchläufen über das gesamte Bild durchführen können.

Da der dominanteste Schritt die euklidische Distanztransformation ist, ist die Gesamtzeitkomplexität der Funktion `calculate_dominance_distance` $O(N)$, das heißt, sie ist proportional zur Gesamtzahl der Pixel in der Höhenkarte. Es ist jedoch zu beachten, dass diese Funktion in der übergeordneten Logik für jeden prominenten Gipfel aufgerufen wird. Wenn P die Anzahl der prominenten Gipfel ist, führt dies zu einer Gesamtkomplexität von $O(P \cdot N)$ für die vollständige Dominanzberechnung aller prominenten Gipfel. Obwohl dies im schlimmsten Fall eine hohe Komplexität bedeuten könnte, ist die Anzahl der prominenten Gipfel (P) in der Praxis typischerweise deutlich geringer als die Gesamtanzahl der Pixel (N). Dies begrenzt die effektive Dauer dieser Operation und macht sie im Kontext der gesamten Gipfelberechnung oft weniger dominierend als andere Schritte.

4.7.5 Berechnung der orografischen Dominanz

Die Funktion `calculate_orographic_dominance` implementiert die Formel für die orografische Dominanz (oD):

$$oD = \frac{P}{H} \cdot 100$$

Wobei P die Prominenz des Gipfels und H die Höhe des Gipfels ist. Eine Division durch Null wird abgefangen, indem für den Fall, dass die Gipfelhöhe 0 beträgt, ebenfalls 0 zurückgegeben wird.

4.7.6 Ergebnis und Evaluierung

Grundsätzlich lässt sich festhalten, dass die Gipfelerkennung effizient und schnell ist – insbesondere durch die Optimierung mit Numba. Durch den Einsatz des Max-Min-Dijkstra-Algorithmus wird zudem eine hohe Genauigkeit bei der Ermittlung der Gipfelpositionen erreicht.

Zur Bewertung der Genauigkeit der Gipfelerkennung wurde eine Karte der Walliser Alpen verwendet. Diese Karte basiert auf dem Copernicus GLO-90 DEM, das mithilfe von Radardaten aus Satellitenaufnahmen erstellt wurde [46]. Die verwendete GeoTIFF-Datei hat eine räumliche Auflösung von etwa 90 m × 90 m, was bedeutet, dass ein Pixel einer Fläche von 8100 m² entspricht.

Die Walliser Alpen wurden als Testregion ausgewählt, da sie zahlreiche prominente Viertausender-Gipfel enthalten, wodurch ein aussagekräftiger Vergleich möglich ist. Als Referenz dient die Gipfelliste der UIAA, welche einen Berggipfel bei einer Mindestprominenz von 30 m definiert. Diese Definition wurde auch für die Evaluierung verwendet. Die Ergebnisse der Erkennung sind in Abbildung 4.6 dargestellt:

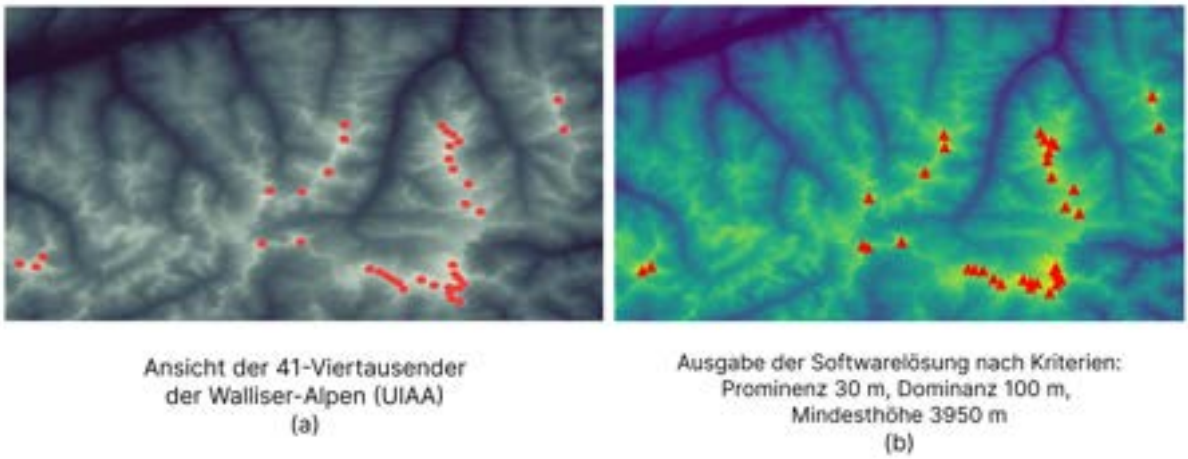

Abbildung 4.6: Vergleich der berechneten Gipfel in der Wallis-Region

Bei (a) sind die von der UIAA gelisteten Gipfel dargestellt und bei (b) die mit der entwickelten Software erkannten. Insgesamt wurden 36 Gipfel erkannt – fünf weniger als in der UIAA-Liste aufgeführt. Die Berechnung dauerte rund 2s.

Zu beachten ist, dass einige der erkannten Gipfel die Viertausender-Marke knapp unterschreiten. Aus diesem Grund wurde auf der rechten Karte eine Mindesthöhe von 3950 m gesetzt, da die Positionen dieser Gipfel mit denen der UIAA-Liste übereinstimmen, auch wenn die Höhe laut DEM niedriger ausfiel. Zusätzlich sind zwei falsch-positive Gipfel zu beobachten. Trotz dieser Differenzen befinden sich nahezu alle erkannten Gipfel an denselben Positionen wie die Referenzpunkte der UIAA. Die nicht erkannten und die falsch-positiven Gipfeln lassen sich durch folgende Ursachen erklären:

Drei der fehlenden Gipfel – die Punta Giordani (5 m Prominenz), die Roccia Nera (20 m) und das Stecknadelhorn (27 m) – unterschreiten die definierte Mindestprominenz von 30 m, sind jedoch dennoch in der UIAA-Liste der Viertausender aufgeführt.

Die übrigen nicht erkannten und falsch-positive Gipfel könnten auf verschiedene Faktoren zurückzuführen sein. Zum einen kann die begrenzte räumliche Auflösung von 90 m × 90 m dazu führen, dass kleinere Geländemerkmale – wie wenig ausgeprägte Gipfel – nicht erfasst werden. Zum anderen kann auch die Genauigkeit der Höhenmessung durch satellitengestützte Radardaten leichte Abweichungen verursachen, was sich auf die exakte Gipfellokalisierung auswirkt.

4.8 Fazit

Das Kapitel 4 beschreibt die Entwicklung einer Softwarelösung zur automatischen Erkennung von Gipfeln. Die Implementierung erfolgt in Python unter Verwendung bewährter Bibliotheken wie Rasterio, NumPy, SciPy, CustomTkinter und Matplotlib. Diese ermöglichen eine verlässliche Geodatenverarbeitung, numerische Berechnungen sowie eine grafische Darstellung der Ergebnisse.

Die Software verarbeitet GeoTIFF-Dateien zuverlässig und identifiziert Gipfel anhand der Parameter wie die Prominenz und Dominanz. Die Analyse basiert auf einem zweistufigen Verfahren, das eine Vorfilterung mittels Bresenham-Algorithmus und eine anschließende Auswertung mit dem Max-Min-Dijkstra-Algorithmus kombiniert. Die Benutzeroberfläche erlaubt das Hochladen von Dateien sowie die Anpassung relevanter Schwellenwerte. Das digitale Höhenmodell und die erkannten Gipfel werden in 2D oder 3D visualisiert. Ergänzend werden die geografischen Koordinaten in das WGS84-System umgerechnet und in einer Tabelle ausgegeben. Zur Leistungsoptimierung kommt Numba zum Einsatz, was die Effizienz rechenintensiver Prozesse deutlich verbessert. Die Genauigkeit der Ergebnisse wurde anhand von Referenzdaten überprüft und weist eine hohe Übereinstimmung auf, mit geringen Abweichungen in der Anzahl der erkannten Gipfel. Die Software ist skalierbar und kann auch große Datensätze verarbeiten, obwohl auf weiterführende Optimierungen wie Datenkompression verzichtet wurde.

Insgesamt erfüllt die entwickelte Lösung alle definierten Anforderungen. Die modulare Struktur und der gezielte Einsatz effizienter Werkzeuge tragen zu einer robusten und leistungsfähigen Umsetzung der automatisierten Gipfelerkennung bei.

5 Zusammenfassung und Ausblick

Dieses Kapitel fasst die wesentlichen Ergebnisse der vorliegenden Arbeit und Funktionen der entwickelten Anwendung zusammen. Anschließend werden mögliche Weiterentwicklungen und zukünftige Forschungsrichtungen aufgezeigt, die sich aus den gewonnenen Erkenntnissen ergeben.

5.1 Zusammenfassung

Die theoretischen Grundlagen der Arbeit umfassen wesentliche Konzepte der Geowissenschaften sowie verschiedene Ansätze zur Gipfelklassifizierung. Digitale Höhenmodelle stellen Geländeinformationen als Rasterdaten dar und entstehen durch Verfahren wie GPS-Vermessung, LIDAR oder Photogrammetrie. Zur objektiven Unterscheidung von Gipfeln dienen Kriterien wie Höhe, Prominenz, Dominanz und orografische Dominanz. Da keine einheitliche Gipfeldefinition existiert, werden verschiedene Klassifikationssysteme wie die von Jurgalski oder der UIAA berücksichtigt.

Die für die Gipfelerkennung eingesetzte Methodik basiert auf einem lokalen Maximum-Filter zur Bestimmung potenzieller Gipfel. Für die Prominenzberechnung kombiniert die Anwendung den Bresenham-Algorithmus für eine grobe Annäherung und den Max-Min-Dijkstra-Algorithmus für die exakte Berechnung. Die Dominanz wird über die euklidische Distanztransformation ermittelt. Die Implementierung davon erfolgt in Python. Eine modulare Architektur trennt dabei die Datenextraktion, Benutzeroberfläche, Geodatenfunktionen und Gipfelanalyse.

Die Arbeit zeigt, dass sich Gipfel automatisiert und effizient aus digitalen Höhendaten extrahieren lassen. Die entwickelte Anwendung kombiniert bewährte

Algorithmen mit modernen Optimierungstechniken und erreicht sowohl hohe Genauigkeit als auch praktikable Laufzeiten.

5.2 Ausblick

Die entwickelte Software bietet verschiedene Ansatzpunkte für weiterführende Forschung und Entwicklung. Eine naheliegende Erweiterung besteht in der Optimierung für Höhendaten mit hoher Auflösung, da moderne Erfassungsmethoden Auflösungen von wenigen Metern oder sogar Dezimetern ermöglichen. Die Erweiterung auf große Datenmengen und die Verarbeitung größerer Bereiche erfordert weitere Optimierungen der Algorithmen und die Entwicklung verteilter Berechnungsansätze. Cloud-Computing-Plattformen und parallele Verarbeitung sind mögliche Ansätze, die die Skalierbarkeit der Methode erheblich verbessern könnten.

Die Integration alternativer Klassifikationssysteme stellt einen weiteren Entwicklungsbereich dar. Neben den implementierten Kriterien können zusätzliche Parameter wie geologische Eigenschaften, Hangneigung oder Geländeform die Gipfelklassifizierung erweitern. Zusätzlich könnte in Erwägung gezogen werden, Technologien wie Machine Learning einzusetzen um, komplexe Zusammenhänge zwischen verschiedenen topografischen Merkmalen zu erfassen und automatisch zu gewichten.

Die Kombination mit anderen geowissenschaftlichen Daten wie Satellitenbild-Zeitreihen oder seismischen Messungen eröffnet zudem neue Möglichkeiten für die Erforschung von Landschaftsveränderungen. Die entwickelte Methodik kann als Grundlage für komplexere Analyseverfahren dienen, die verschiedene Datenquellen integrieren und ganzheitliche Betrachtungen der Erdoberfläche ermöglichen.

Literatur

[1] T. Hahn, *Entwicklung eines automatischen Gipfelfinder bei Topografischen Karten*, 2024.

[2] L. Müller, *Automatischer Gipfelfinder bei topografischen Karten*, 2024.

[3] S. Rudolph, *Entwicklung einer Methodik zur Detektion von Berggipfeln aus einer digitalen topographischen Karte*, 2024.

[4] E. Jurgalski, «Das erweiterte Prominenzkonzept,» de, 2015.

[5] C. A. Manduca und K. A. Kastens, «Geoscience and geoscientists: Uniquely equipped to study Earth,» en, in *Earth and Mind II: A Synthesis of Research on Thinking and Learning in the Geosciences*, Geological Society of America, 2012, ISBN: 978-0-8137-2486-7. DOI: 10.1130/2012.2486(01). Adresse: https://pubs.geoscienceworld.org/books/book/643/chapter/3806356 (besucht am 07. 06. 2025).

[6] *Was ist Geographie? | Eberhard Karls Universität Tübingen*, 2025. Adresse: https://uni-tuebingen.de/fakultaeten/mathematisch-naturwissenschaftliche-fakultaet/fachbereiche/geowissenschaften/studium/schuelerinformation/geographie/was-ist-geographie/ (besucht am 11. 06. 2025).

[7] D. Angermann, R. Pail, F. Seitz und U. Hugentobler, «Die Vermessung der Erde im Wandel der Zeit,» de, in *Mission Erde: Geodynamik und Klimawandel im Visier der Satellitengeodäsie*, D. Angermann, R. Pail, F. Seitz und U. Hugentobler, Hrsg., Berlin, Heidelberg: Springer, 2021, S. 11–55, ISBN: 978-3-662-62338-1. DOI: 10.1007/978-3-662-62338-1_2. Adresse: https://doi.org/10.1007/978-3-662-62338-1_2 (besucht am 11. 06. 2025).

[8] B. Markoski, «Basic Principles of Topography,» en, in *Basic Principles of Topography*, B. Markoski, Hrsg., Cham: Springer International Publishing, 2018, S. 1–24, ISBN: 978-3-319-72147-7. DOI: 10.1007/978-3-319-72147-7_1. Adresse: https://doi.org/10.1007/978-3-319-72147-7_1 (besucht am 11. 06. 2025).

[9] M. H. P. Ambaum und W. T. M. Verkley, «Orography in a Contour Dynamics Model of Large-Scale Atmospheric Flow,» en, Aug. 1995, Section: Journal of the Atmospheric Sciences, ISSN: 1520-0469. Adresse: https://journals.ametsoc.org/view/journals/atsc/52/15/1520-0469_1995_052_2643_oiacdm_2_0_co_2.xml (besucht am 11. 06. 2025).

[10] J. Bollmann und W. G. Koch, «Das neue Lexikon der Kartographie und Geomatik,» *KN - Journal of Cartography and Geographic Information*, Jg. 52, Nr. 2, S. 66–71, März 2002, ISSN: 2524-4965. DOI: 10.1007/BF03544878. Adresse: https://doi.org/10.1007/BF03544878.

[11] L. Croneborg, K. Saito, M. Matera, D. McKeown und J. van Aardt, «DIGITAL ELEVATION MODELS,» en, 2015.

[12] M. Meybeck, P. Green und C. Vörösmarty, «A New Typology for Mountains and Other Relief Classes,» *Mountain Research and Development*, Jg. 21, Nr. 1, S. 34–45, Feb. 2001, Publisher: International Mountain Society, ISSN: 0276-4741, 1994-7151. DOI: 10.1659/0276-4741(2001)021[0034:ANTFMA]2.0.CO;2. Adresse: https://bioone.org/journals/mountain-research-and-development/volume-21/issue-1/0276-4741_2001_021_0034_ANTFMA_2.0.CO_2/A-New-Typology-for-Mountains-and-Other-Relief-Classes/10.1659/0276-4741(2001)021%5B0034:ANTFMA%5D2.0.CO;2.full (besucht am 11. 06. 2025).

[13] H. Cycon, «Relative Extremwerte von Funktionen im Mehrdimensionalen,» de, in *Mathematik visuell und interaktiv: für Ingenieure und Naturwissenschaftler*, H. Cycon, Hrsg., Wiesbaden: Springer Fachmedien, 2020, S. 299–327, ISBN: 978-3-658-30245-0. DOI: 10.1007/978-3-658-30245-0_16. Adresse: https://doi.org/10.1007/978-3-658-30245-0_16 (besucht am 11. 06. 2025).

[14] S. v. Innstädten, *Allgemeine Orographie*, de. Verlag nicht ermittelbar, 1873, Google-Books-ID: ONXTJdTap8EC.

[15] O. Lanser, «Paß-Bezeichnungen in den Alpen,» in 1951. Adresse: https://www.zobodat.at/pdf/VeroeffFerd_31_0493-0500.pdf (besucht am 11. 06. 2025).

[16] P. L. Guth, A. Van Niekerk, C. H. Grohmann u. a., «Digital Elevation Models: Terminology and Definitions,» en, *Remote Sensing*, Jg. 13, Nr. 18, S. 3581, Sep. 2021, ISSN: 2072-4292. DOI: 10.3390/rs13183581. Adresse: https://www.mdpi.com/2072-4292/13/18/3581 (besucht am 02. 06. 2025).

[17] R. Roblero-Hidalgo, J. Flores-Velázquez, J. Chávez-Morales und A. Reyes-Ramírez, «Reliability assessment of three topographic methods for generating digital elevation models (DEMs),» es, *Ingeniería Agrícola y Biosistemas*, Jg. 12, Nr. 2, S. 159–171, 2020, ISSN: 20073925, 20074026. DOI: 10.5154/r.inagbi.2020.01.005. Adresse: https://revistas.chapingo.mx/inagbi/?section=articles&subsec=issues&numero=286&articulo=2689 (besucht am 02. 06. 2025).

[18] J. Gong, «Effects of Various Factors on the Accuracy of DEMs: An Intensive Experimental Investigation,» en, *PHOTOGRAMMETRIC ENGINEERING*, 2000.

[19] V. Pászto, A. Redecker, K. Macků, C. Jürgens und N. Moos, «Data Sources,» en, in *Spationomy*, V. Pászto, C. Jürgens, P. Tominc und J. Burian, Hrsg., Cham: Springer International Publishing, 2020, S. 3–38, ISBN: 978-3-030-26625-7. DOI: 10.1007/978-3-030-26626-4_1. Adresse: http://link.springer.com/10.1007/978-3-030-26626-4_1 (besucht am 02. 06. 2025).

[20] N. Ritter und M. Ruth, «The GeoTiff data interchange standard for raster geographic images,» en, *International Journal of Remote Sensing*, Jg. 18, Nr. 7, S. 1637–1647, Mai 1997, ISSN: 0143-1161, 1366-5901. DOI: 10.1080/014311697218340. Adresse: https://www.tandfonline.com/doi/full/10.1080/014311697218340 (besucht am 02. 06. 2025).

[21] S. S. Mahammad und R. Ramakrishnan, «GeoTIFF - A standard image file format for GIS applications,» en, *Image Processing*, 2003.

[22] D. D. Aldus, *TIFF Revision 6.0*, 1992.

[23] M. P. Google, *GeoTIFF-Dateien | Solar API*, de-x-mtfrom-en, 2025. Adresse: https://developers.google.com/maps/documentation/solar/geotiff?hl=de (besucht am 02. 06. 2025).

[24] M. Mills, *What data can be stored in a GeoTIFF file?* en-US, Juni 2022. Adresse: https://www.bluemarblegeo.com/blog/about-geotiff-format/ (besucht am 04. 06. 2025).

[25] W. Leonhard, *Thehighrisepages - Hochhäuser und Berge / Highrises and Mountains*, 1999. Adresse: https://www.thehighrisepages.de/ (besucht am 02. 06. 2025).

[26] R. N. Torres, F. Milani und P. Fraternali, «Algorithms for Mountain Peaks Discovery: A Comparison,» en, 2019.

[27] R. C. Gonzalez und R. E. Woods, *Digital image processing*, en, Fourth edition, global edition. New York: Pearson, 2018, ISBN: 978-0-13-335672-4.

[28] A. Zingl, «Rasterizing Curves,» en, 2012.

[29] M. V. Lossow, «A min-max version of Dijkstra's algorithm with application to perturbed optimal control problems,» en, *PAMM*, Jg. 7, Nr. 1, S. 4 130 027–4 130 028, Dez. 2007, ISSN: 1617-7061, 1617-7061. DOI: 10.1002/pamm.200700646. Adresse: https://onlinelibrary.wiley.com/doi/10.1002/pamm.200700646 (besucht am 02. 06. 2025).

[30] L. Buddrus, «Bottleneck Problems in Graphs,» en,

[31] *Finding the Maximum-Minimum Capacity for a Node in a Graph | Baeldung on Computer Science*, en-US, Mai 2020. Adresse: https://www.baeldung.com/cs/graph-max-min-node-capacity (besucht am 09. 06. 2025).

[32] P. Bankhead, *Image transforms — Introduction to Bioimage Analysis*, 2022. Adresse: https://bioimagebook.github.io/chapters/2-processing/6-transforms/transforms.html (besucht am 02. 06. 2025).

[33] E. Rouault, F. Warmerdam, K. Schwehr u. a., *GDAL*, Mai 2025. DOI: 10.5281/ZENODO.5884351. Adresse: https://zenodo.org/doi/10.5281/zenodo.5884351 (besucht am 03. 06. 2025).

[34] L. Prechelt, «An empirical comparison of C, C++, Java, Perl, Python, Rexx, and Tcl for a search/string-processing program,» en,

[35] *Numba: A High Performance Python Compiler*. Adresse: https://numba.pydata.org/ (besucht am 03. 06. 2025).

[36] *Rasterio: access to geospatial raster data — rasterio 1.4.3 documentation*. Adresse: https://rasterio.readthedocs.io/en/stable/index.html (besucht am 03. 06. 2025).

[37] *pyproj 3.7.1 documentation*. Adresse: https://pyproj4.github.io/pyproj/stable/ (besucht am 03. 06. 2025).

[38] *NumPy documentation — NumPy v2.2 Manual*. Adresse: https://numpy.org/doc/stable/ (besucht am 03. 06. 2025).

[39] *SciPy documentation — SciPy v1.15.3 Manual*. Adresse: https://docs.scipy.org/doc/scipy/ (besucht am 03. 06. 2025).

[40] S. Van Der Walt, J. L. Schönberger, J. Nunez-Iglesias u. a., «scikit-image: image processing in Python,» en, *PeerJ*, Jg. 2, e453, Juni 2014, ISSN: 2167-8359. DOI: 10.7717/peerj.453. Adresse: https://peerj.com/articles/453 (besucht am 03. 06. 2025).

[41] *heapq — Heap queue algorithm*, en. Adresse: https://docs.python.org/3/library/heapq.html (besucht am 03. 06. 2025).

[42] *PyQt - Python Wiki*. Adresse: https://wiki.python.org/moin/PyQt (besucht am 03. 06. 2025).

[43] *tkinter — Python interface to Tcl/Tk*, en. Adresse: https://docs.python.org/3/library/tkinter.html (besucht am 03. 06. 2025).

[44] *Official Documentation And Tutorial | CustomTkinter*, en. Adresse: https://customtkinter.tomschimansky.com/ (besucht am 03. 06. 2025).

[45] *Matplotlib — Visualization with Python*. Adresse: https://matplotlib.org/ (besucht am 03. 06. 2025).

[46] *OpenTopography - Copernicus GLO-90 Digital Elevation Model*. Adresse: https://portal.opentopography.org/raster?opentopoID=OTSDEM.032021.4326.1 (besucht am 04. 06. 2025).